

Formularze w Java

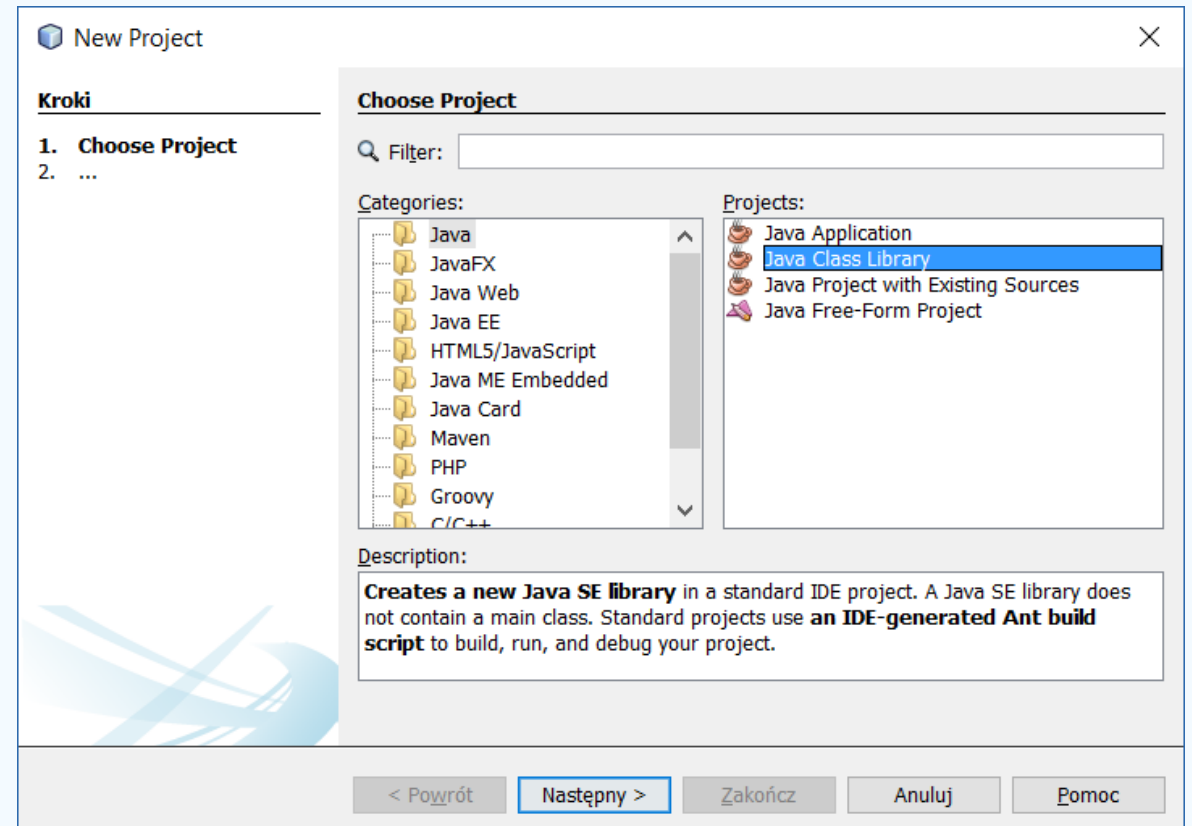
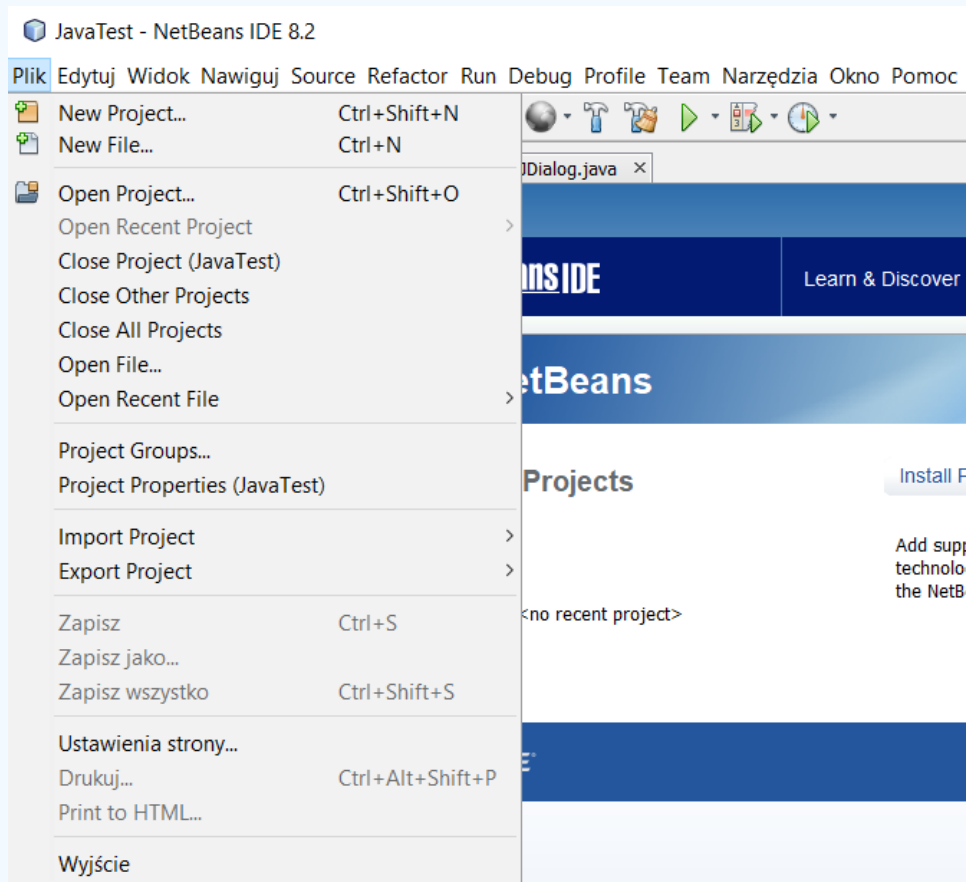
Zbigniew Marszalek

Komponenty interfejsu użytkownika Swing

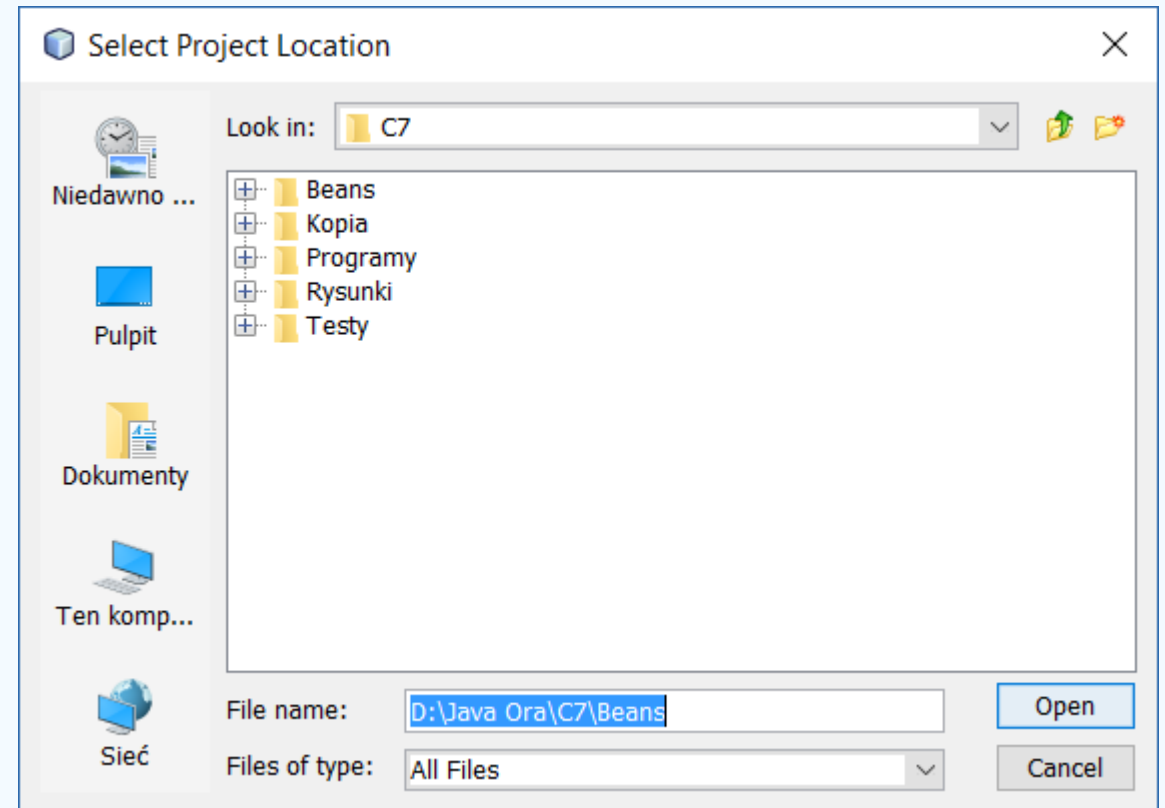
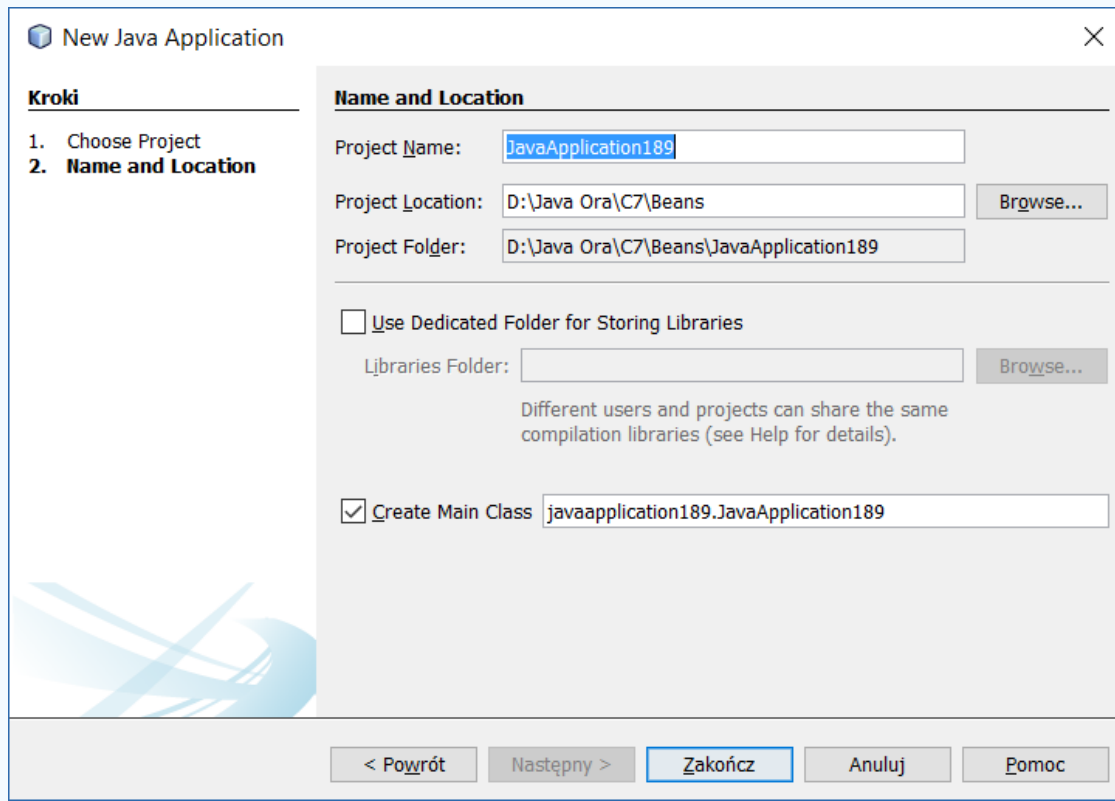
Wykład ten omawia podstawowe elementy interfejsu użytkownika zestawu Swing. Net Beans IDE umożliwia łatwy sposób wykorzystania popularnych komponentów takich jak pola tekstowe, etykiety, przyciski, paski przewijania oraz menu. Daj również możliwość pracy z narzędziami zarządzającym układem graficznym , dzięki któremu można organizować położenie komponentów w oknie, niezależnie od wyglądu danego interfejsu użytkownika. Przedstawmy obecnie sposób wykonania projektu w programie Net Beans IDE.

Do budowy zawansowanych aplikacji okienkowych służy zestaw AWT (ang. Abstract Window Toolkit). AWT składa się z wielu klas i metod, za pomocą których można tworzyć okna i zarządzać nimi. Zestaw narzędzi AWT stanowi także podstawę, na której zbudowano komponenty Swing.

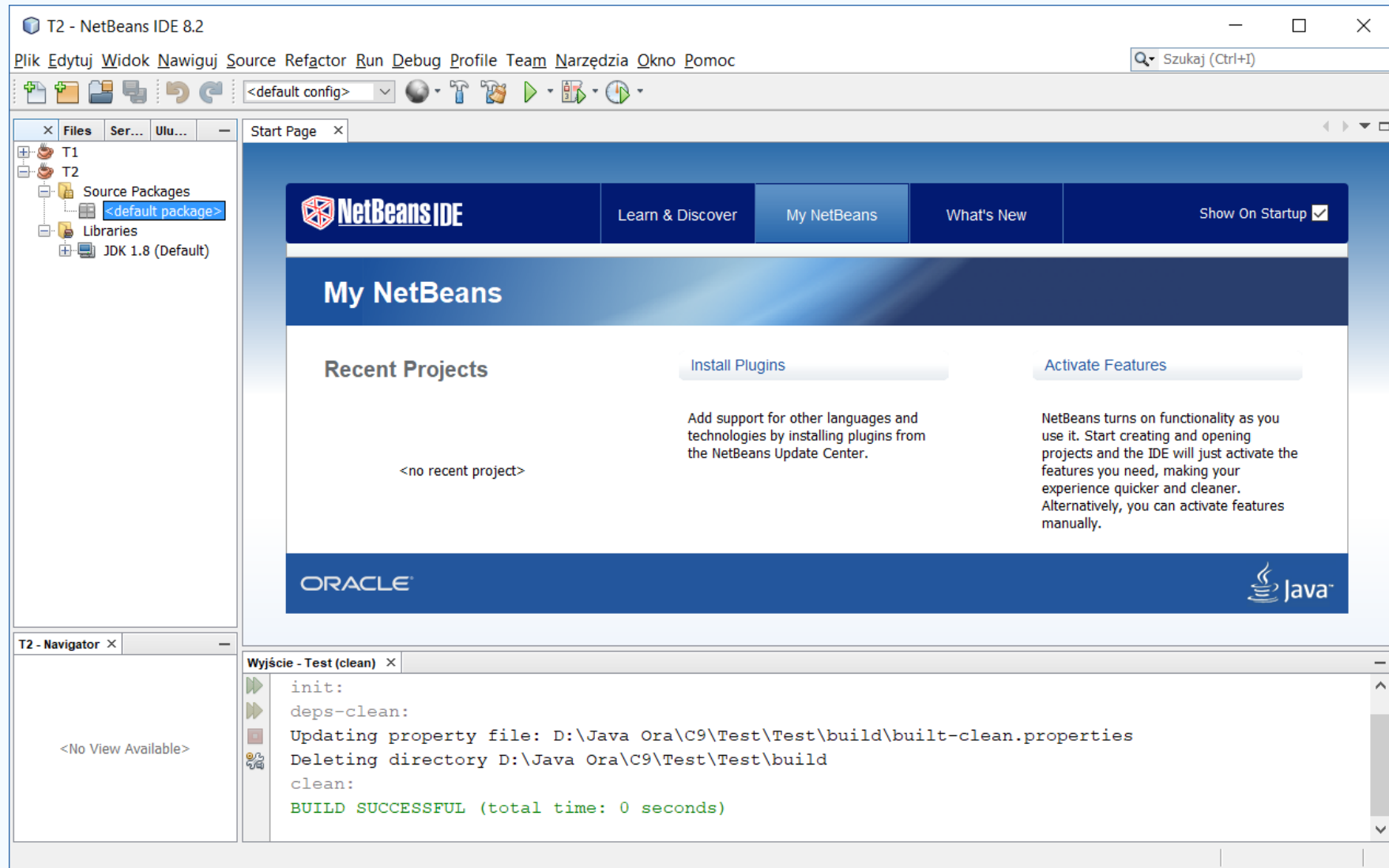
Budujemy nowy projekt w programie Net Beans IDE klikają w zakładce Plik pozycję New Project. (Ctrl+Shift+N). Można to zrobić prościej, kliknąć w menu Net Beans w ikonę Nowy Projekt. W ukazującym się oknie wybieramy pozycję Java oraz Java Class Library i klikamy w przycisk Następny>.



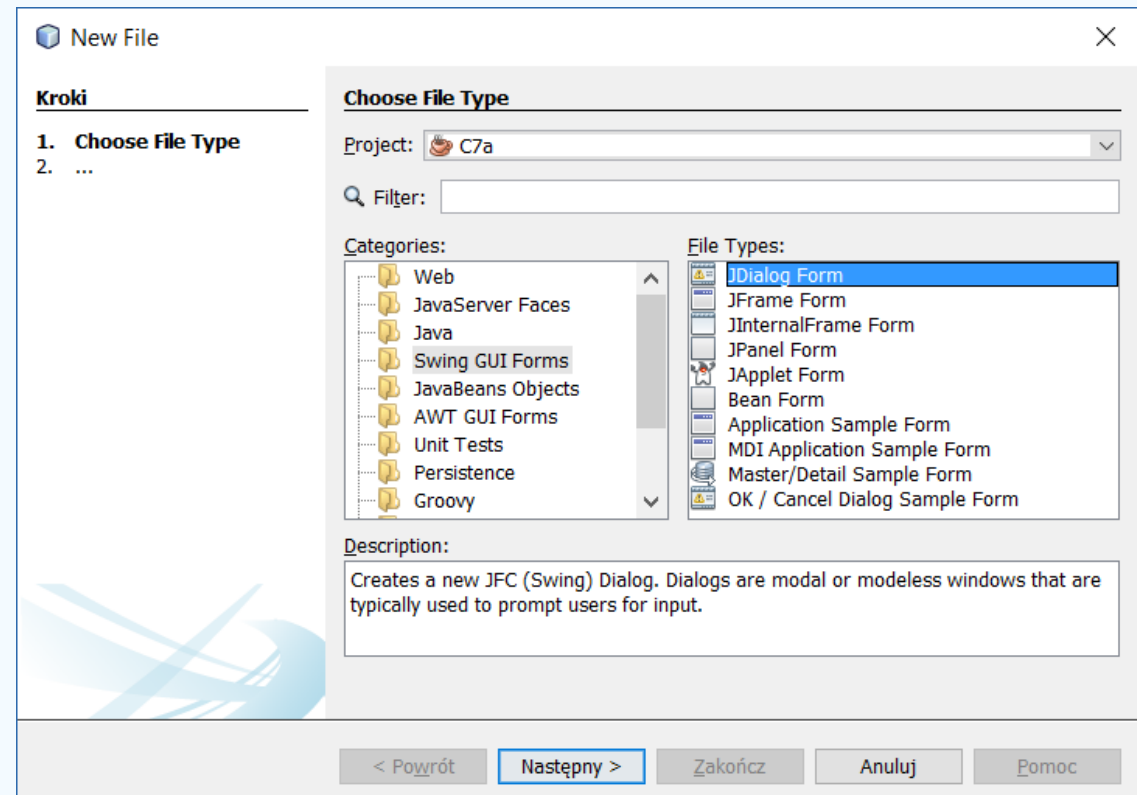
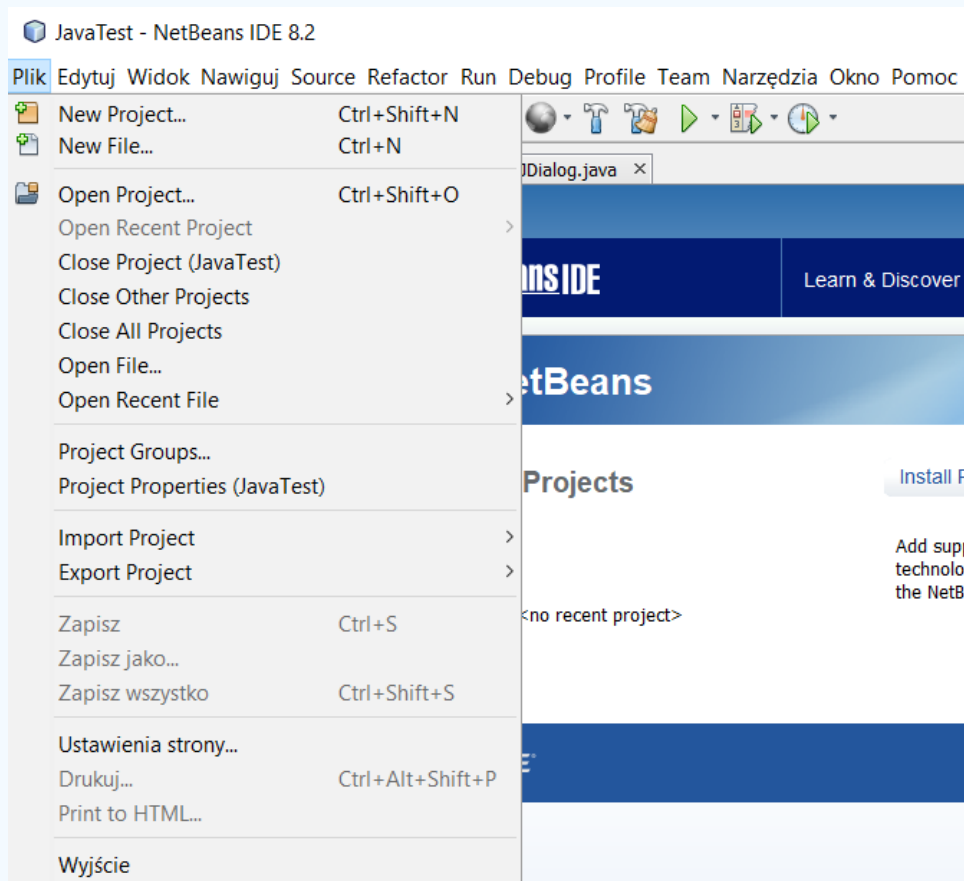
W pojawiającym się oknie wpisujemy nazwę projektu i za pomocą przycisku Browse wybieramy lokalizację naszego projektu. Następnie klikamy w zakładkę Zakończ.



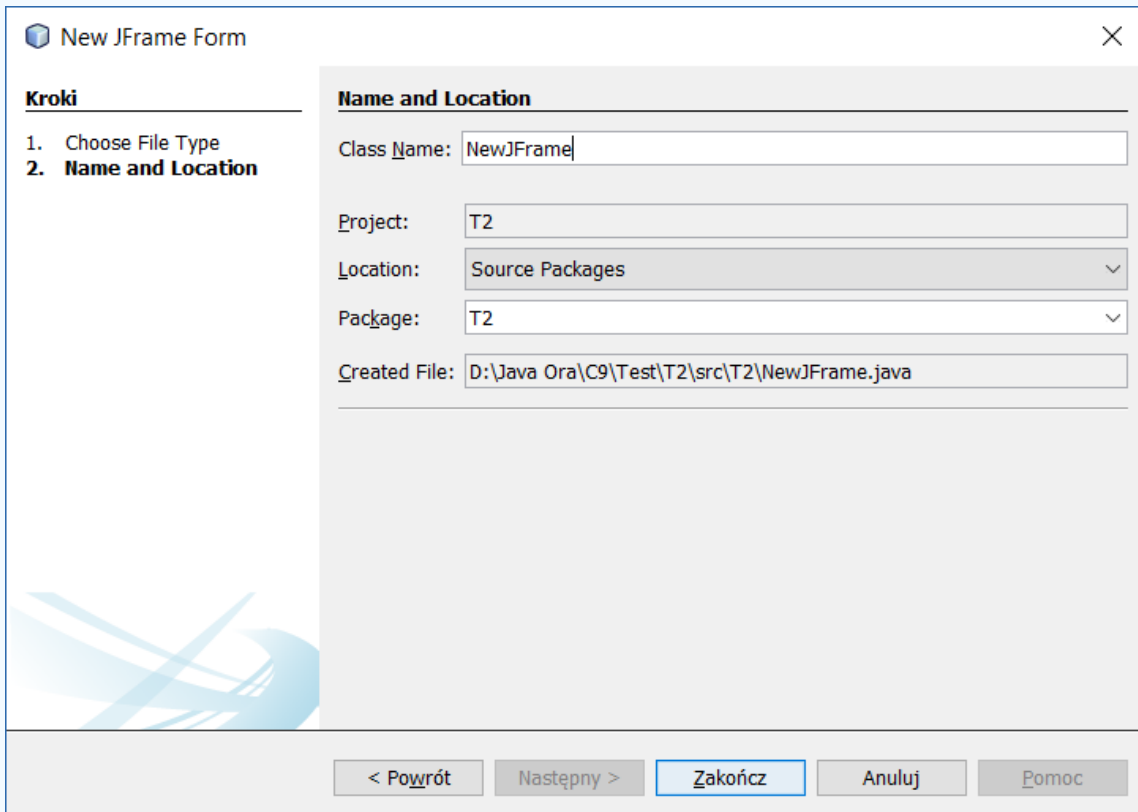
Okno projektu aplikacji w języku Java



W ten sposób otrzymujemy szablon pustego projektu w języku Java. Do otrzymanego projektu dodajmy kolejny element wybierając w zakładce Plik pozycję New File (Ctrl+N) lub klikając w ikonę reprezentującą nowy plik. W ukazującym się oknie wybieramy pozycję Swing GUI Forms i JDialog Forms. W wyświetlonym oknie klikamy w przycisk Następny>.



W pojawiającym się oknie wpisujemy nazwę klasy i pakietu. Za pomocą przycisku Browse wybieramy lokalizację naszego projektu. Następnie klikamy w zakładkę Zakończ.



New JFrame Form

Kroki

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name: NewJFrame

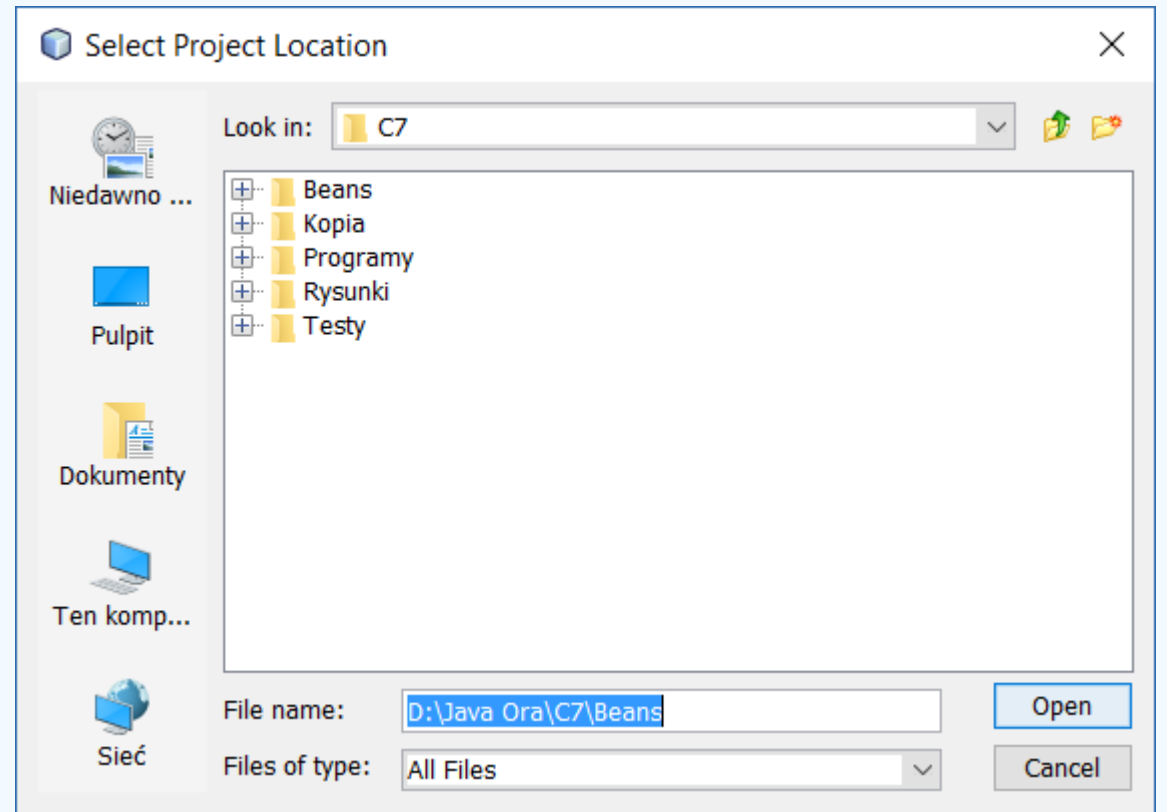
Project: T2

Location: Source Packages

Package: T2

Created File: D:\Java Ora\C9\Test\T2\src\T2\NewJFrame.java

< Powrót Następny > **Zakończ** Anuluj Pomoc



Select Project Location

Look in: C7

Niedawno ...

- + Beans
- + Kopia
- + Programy
- + Rysunki
- + Testy

Pulpit

Dokumenty

Ten komp...

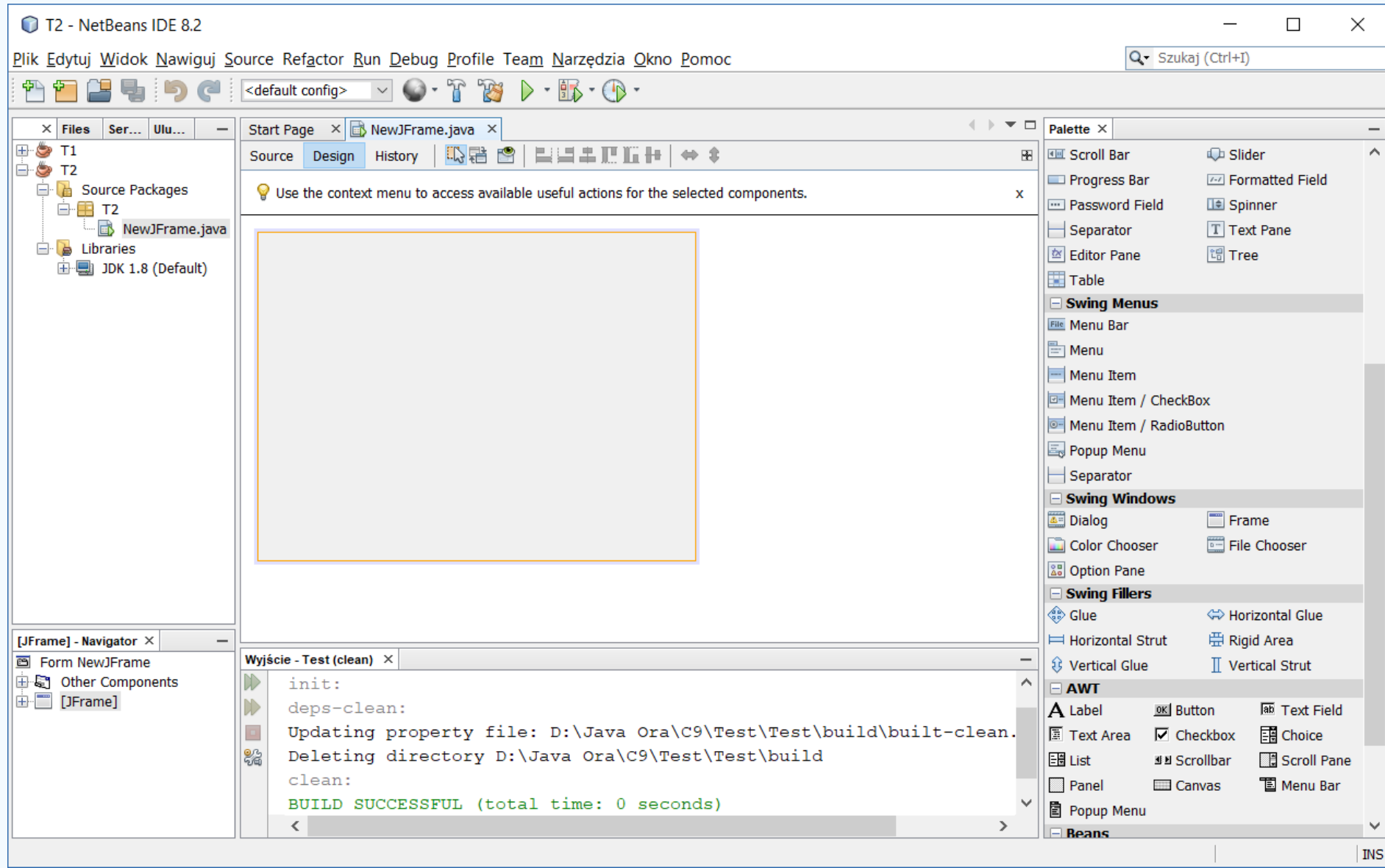
Sieć

File name: D:\Java Ora\C7\Beans

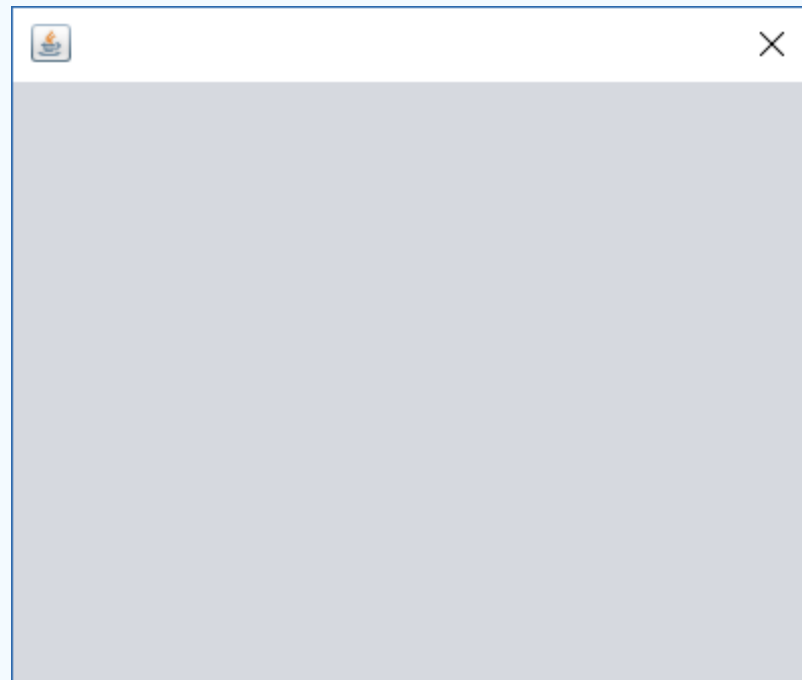
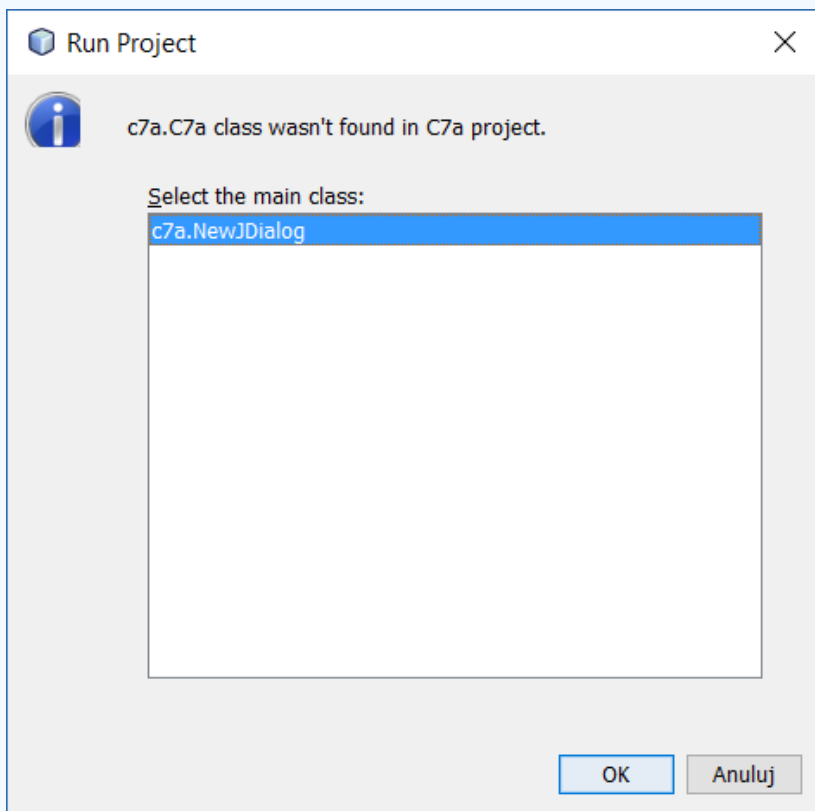
Files of type: All Files

Open Cancel

Projekt formularza w NetBeans



Uruchamiamy nasz projekt wybierając w zakładce Run pozycję Run project (F6) lub klikając w menu Net Beans ikonę Run i wyskakującym oknie przycisk OK. Zostaje wyświetlony pusty formularz naszej aplikacji



```
package c7a;
public class NewJDialog
    extends javax.swing.JDialog {
    public NewJDialog
        (java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
    }

    /**
     * This method is called from within the
     * constructor to initialize the form.
     * WARNING: Do NOT modify this code.
     * The content of this method is always
     * regenerated by the Form Editor.
     */
}
```

```
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed"
// desc="Generated Code">
//GEN-BEGIN: initComponents
private void initComponents() {
    setDefaultCloseOperation
(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

    javax.swing.GroupLayout layout =
new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup
(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(
                Short.MAX_VALUE)
    );
```

```
layout.setVerticalGroup(  
    layout.createParallelGroup  
        (javax.swing.GroupLayout.Alignment.LEADING)  
            .addGap(0, 300, Short.MAX_VALUE)  
);  
  
pack();  
} // </editor-fold> // GEN-END: initComponents  
  
/**  
 * @param args the command line arguments  
 */  
public static void main(String args[]) {  
    try {
```

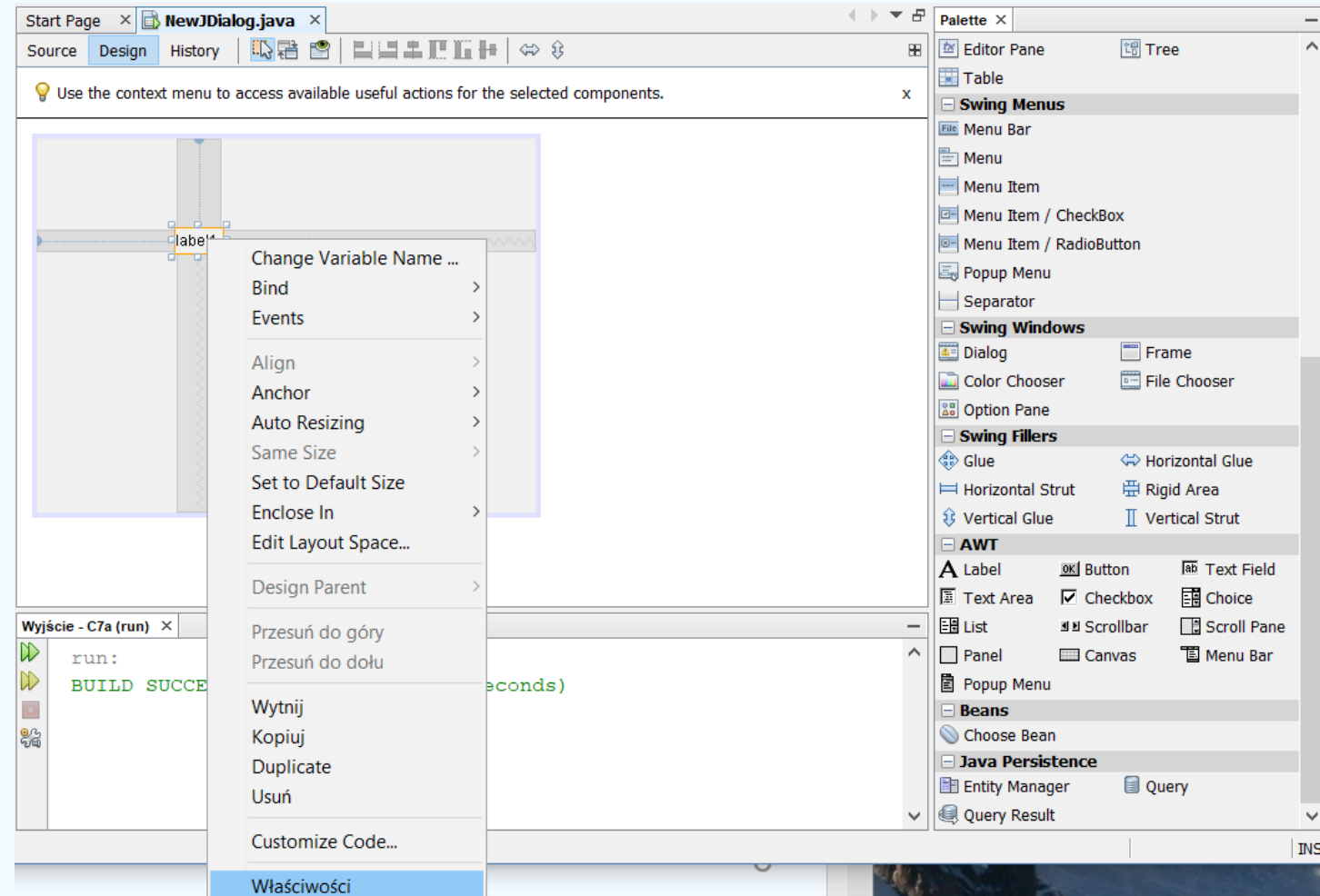
```
    for
        (javax.swing.UIManager.LookAndFeelInfo info :
 javax.swing.UIManager.getInstalledLookAndFeels())
    {
        if ("Nimbus".equals(info.getName())) {
            javax.swing.UIManager.setLookAndFeel
                (info.getClassName());
            break;
        }
    }
} catch (ClassNotFoundException ex) {
    java.util.logging.Logger.getLogger
        (NewJDialog.class.getName()).log
        (java.util.logging.Level.SEVERE, null, ex);
}
```

```
} catch (InstantiationException ex) {  
    java.util.logging.Logger.getLogger  
        (NewJDialog.class.getName()).log  
        (java.util.logging.Level.SEVERE, null, ex);  
}  
catch (IllegalAccessException ex) {  
    java.util.logging.Logger.getLogger  
        (NewJDialog.class.getName()).log  
        (java.util.logging.Level.SEVERE, null, ex);  
}  
catch  
(javax.swing.UnsupportedLookAndFeelException ex) {  
    java.util.logging.Logger.getLogger  
        (NewJDialog.class.getName()).log  
        (java.util.logging.Level.SEVERE, null, ex);  
}  
  
//</editor-fold>
```

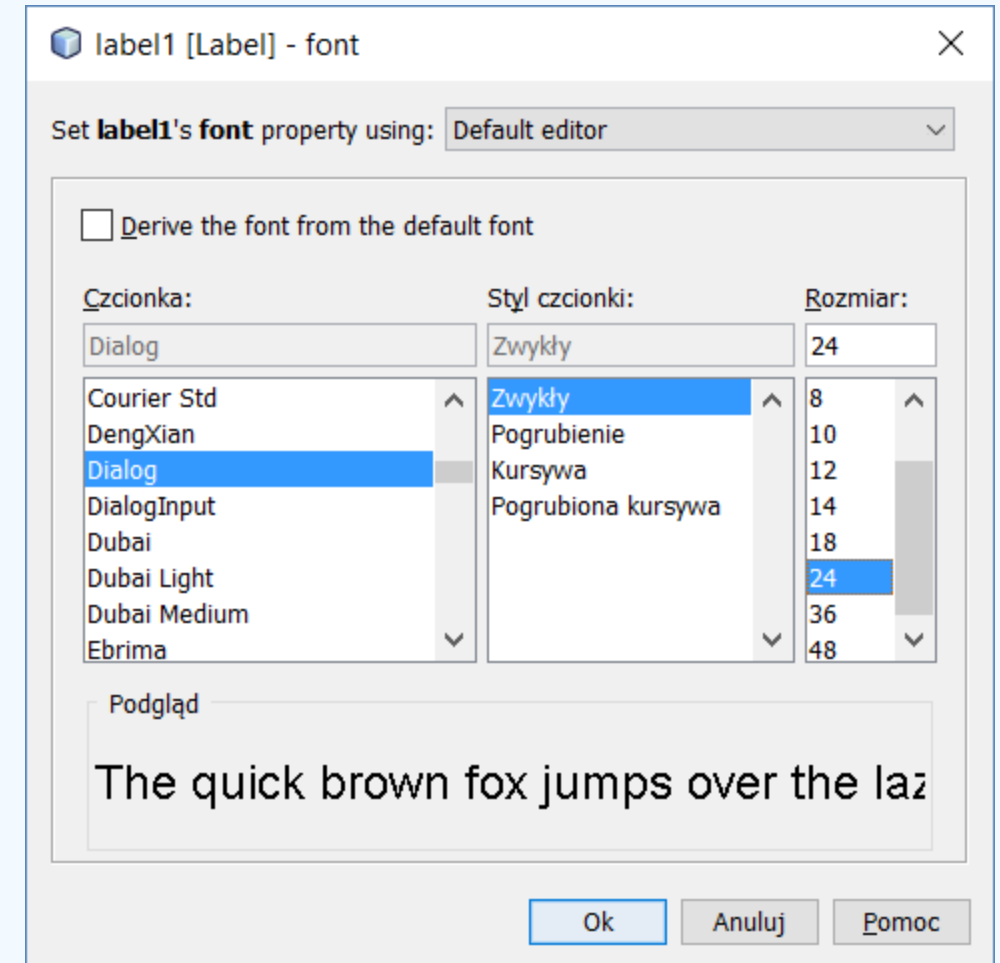
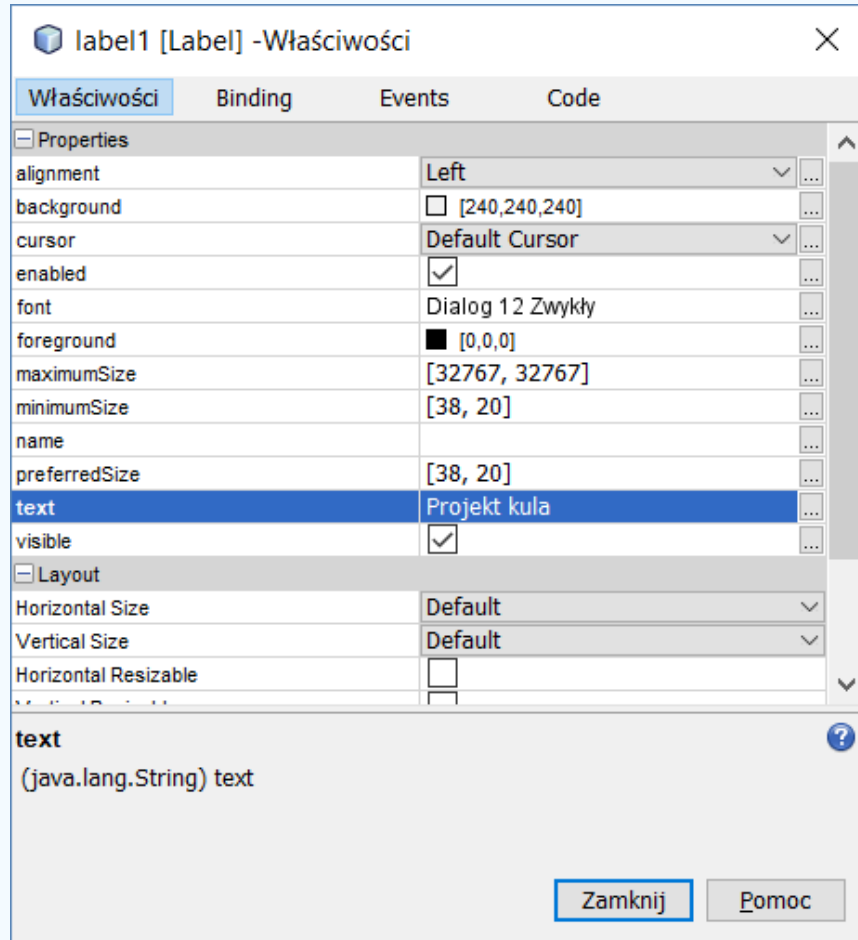
```
/* Create and display the dialog */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        NewJDialog dialog = new
        NewJDialog(new javax.swing.JFrame(), true);
        dialog.addWindowListener
        (new java.awt.event.WindowAdapter() {
            @Override
            public void windowClosing
            (java.awt.event.WindowEvent e) {
                System.exit(0);
            }
        });
        dialog.setVisible(true);
    }
});
}}
```

Formatka etykiety

Z palety AWT wybieramy ikonę Label i za pomocą myszy klikając lewym klawiszem przeciągamy na powierzchnię formularza. Klikamy prawym klawiszem w etykietę i w ukazującym się oknie wybieramy zakładkę Właściwości.

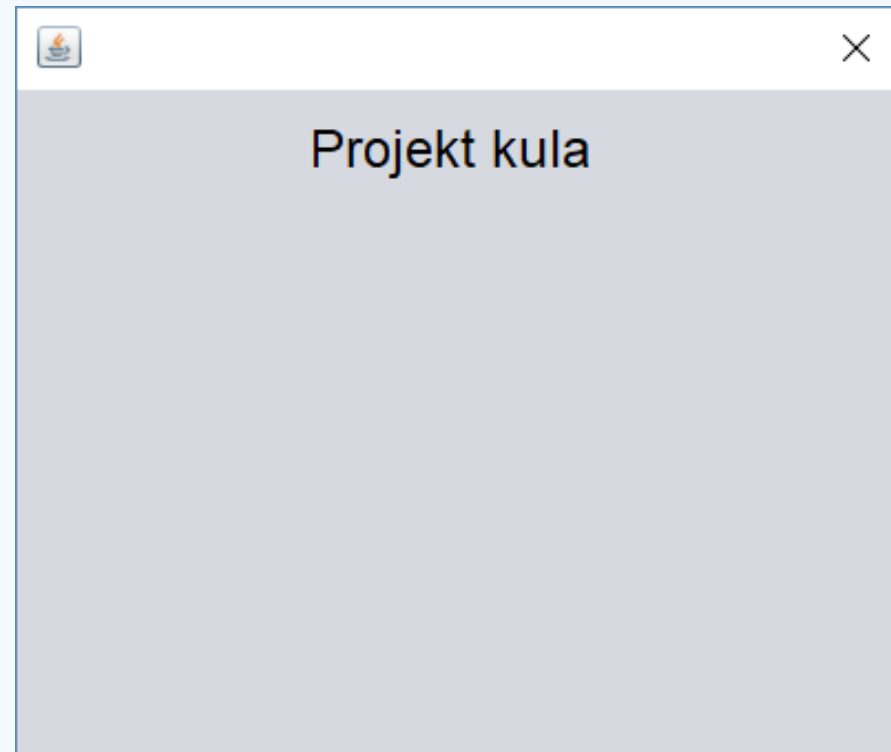


We właściwościach etykiety zmieniamy pole text label1 na Projekt kula. Następnie dostosowujemy wielkość czcionki w wyskakującym oknie pola Font. Następnie zamykamy właściwości klikając w przycisk Zamknij.



Uruchomienie projektu

Projekt uruchamiamy wybierając Run projekt w zakładce Run menu Net Beans (F6) lub bezpośrednio klikając w ikonę uruchamiania projektu.



Spróbujmy teraz zbudować aplikację obliczającą objętość kuli. Do tego celu potrzebujemy okienek umożliwiający wprowadzenie promienia.

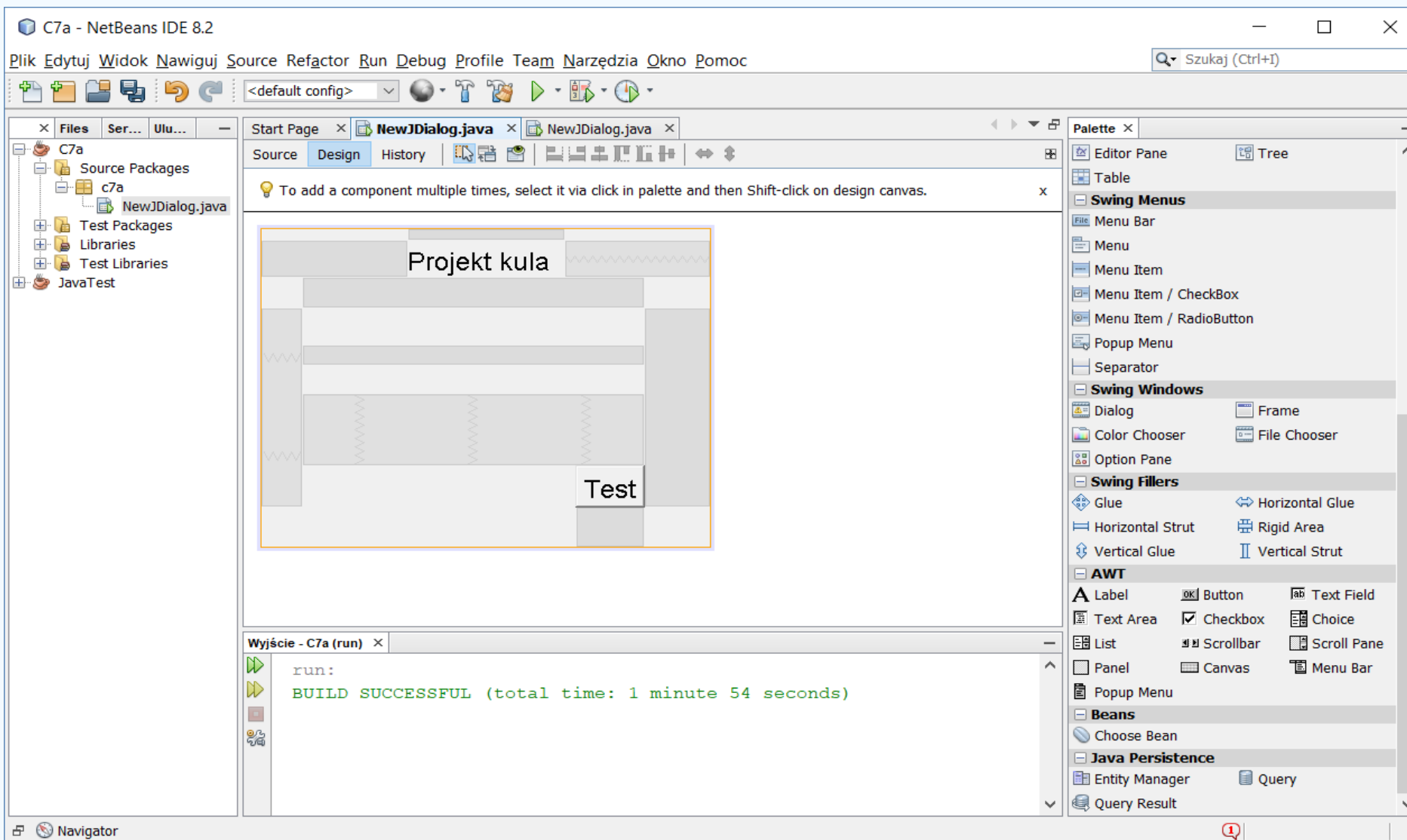
Promień jest liczbą typu rzeczywistego, do wprowadzenia której możemy wykorzystać okno dialogowe służące do wpisywania danych.

Wywołanie metody jest następujące:

String JOptionPane.showInputDialog(**String** *message*)

Następnie przeczytany łańcuch powinniśmy skonwertować na liczbę rzeczywistą. Ponieważ chcemy podawać liczbę w interfejsie polskim wykorzystamy do tego celu skaner, który czyta poprawnie liczby typu double zapisane w konwencji polskiej. Po dodaniu do projektu dwóch kolejnych etykiet i przycisku, generujemy zdarzenie kliknięcia w przycisk dwukrotnie klikając w przycisk na projekcie formularza. Wpisujemy kod i nasz projekt wygląda jak na rysunku:

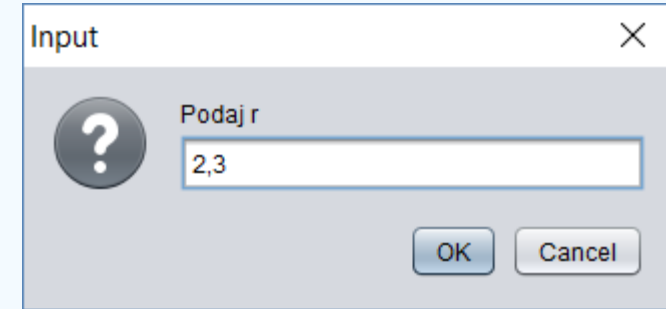
Widok projektu



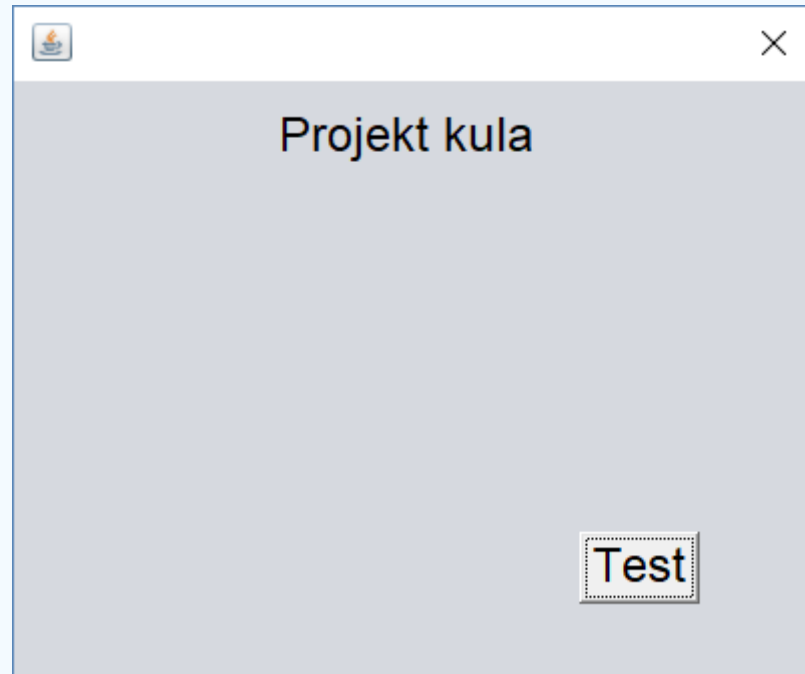
```
private void button1ActionPerformed
    (java.awt.event.ActionEvent evt) {
//GEN-FIRST:event_button1ActionPerformed
    label2.setText("");
    label3.setText("");
    try{
String s = JOptionPane.showInputDialog("Podaj r");
    Scanner in = new Scanner(s);
    double r = in.nextDouble();
    String s1 = String.format("r = %g", r);
    label2.setText(s1);
    }
    catch (Exception e) {
        label3.setText("Błąd konwersji liczby");
    }
} //GEN-LAST:event_button1ActionPerformed
```

Test działania programu

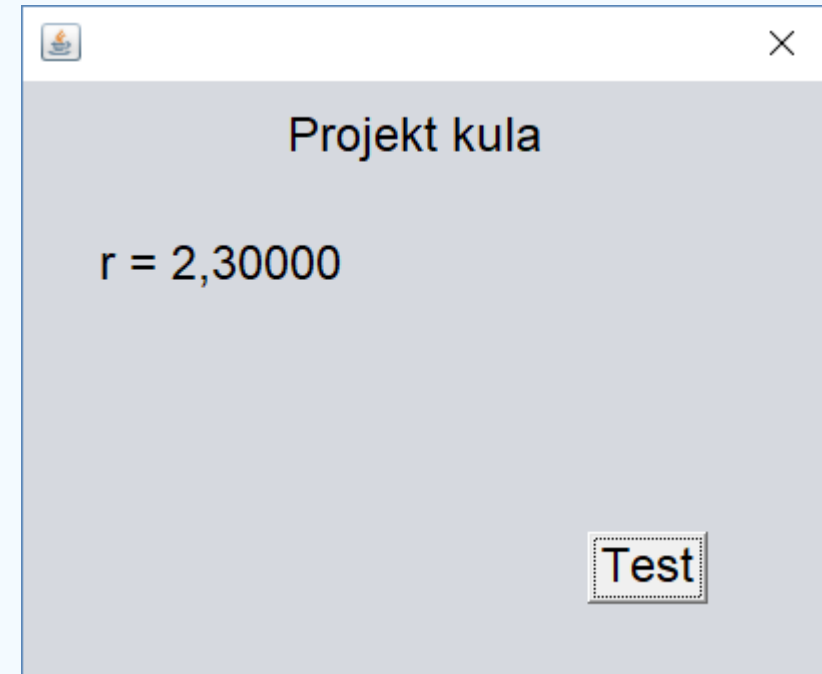
Po uruchomieniu projektu wpisujemy w wyskakującym oknie promień kuli. Po zatwierdzeniu danej wpisanej w polu okna zostaje wyświetlona liczb w formularzu głównym.



A small dialog box titled "Input" with a close button (X) in the top right corner. It contains a question mark icon on the left, the text "Podaj r" above a text input field, and "OK" and "Cancel" buttons at the bottom right. The input field contains the value "2,3".

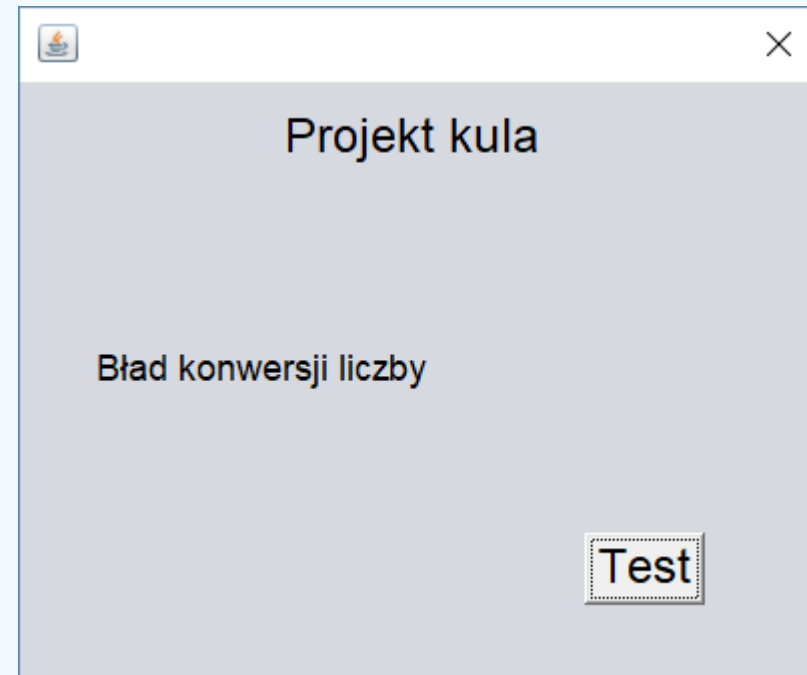
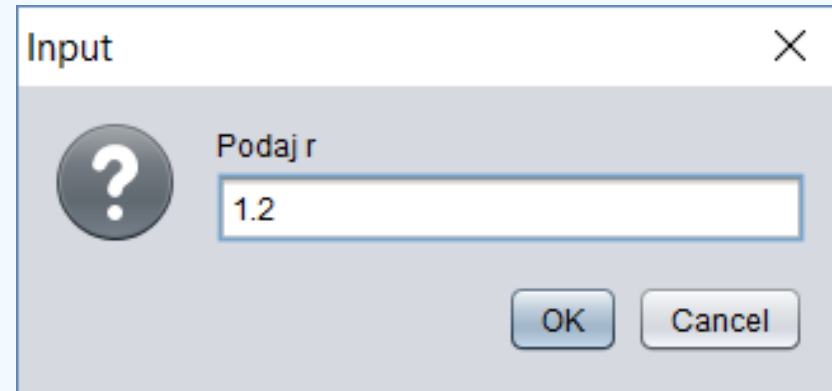
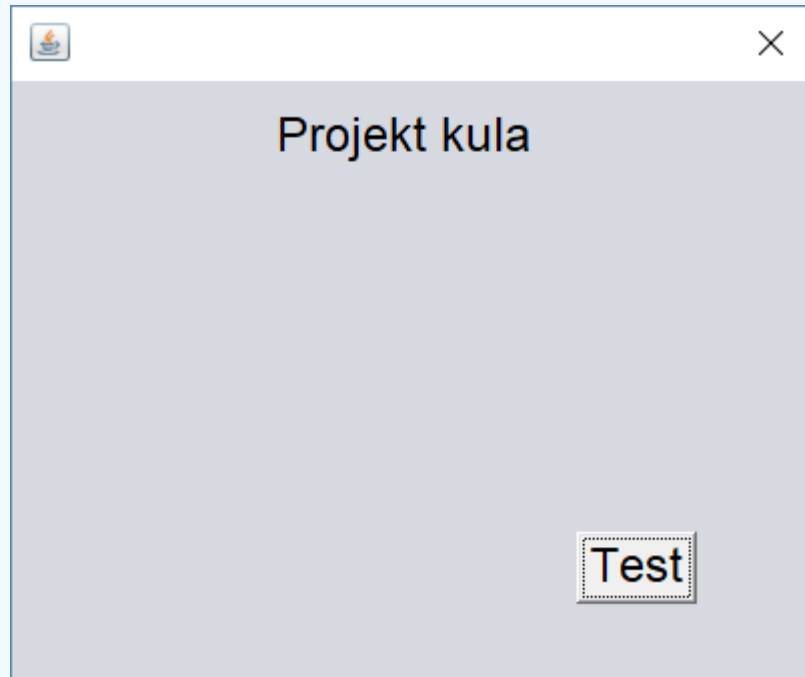


The main application window titled "Projekt kula" with a close button (X) in the top right corner. It has a light gray background and a "Test" button in the bottom right corner. The window is empty except for the title and button.



The main application window titled "Projekt kula" with a close button (X) in the top right corner. It has a light gray background and a "Test" button in the bottom right corner. The text "r = 2,30000" is displayed in the center of the window.

Test działania programu



Formularze modalne

Aplikacja składa się formularza podstawowego i formularza utworzonego w odpowiedzi na zdarzenie kliknięcia przycisku.

Należy zauważyć, że do momentu kiedy otwarte jest drugie okno nie mamy dostępu do formularza podstawowego. Takie rozwiązanie, w którym dostęp jest ograniczony do ostatniego otwartego formularza nazywamy formularzem modalnym (ang. modal form). Innym rozwiązaniem jest zapewnienie dostępu do wszystkich otwartych formularzy określanym jako formularz niemodalny (ang. modeless form).

Przy uruchomieniu formularza wywoływana jest metoda , które reaguje na wystąpienie określonego zdarzenia. W prezentowanej aplikacji za przetworzenie informacji kliknięcia przycisku odpowiada metoda *button1ActionPerformed()*.

Projekt objętość kuli

Tworzymy projekt formularza i rozmieszczamy na powierzchni formularza następujące kontrolki:

1. Kontrolkę przycisku. Zmieniamy wyświetlany tekst w kontrolce na „Oblicz”.
2. Trzy kontrolki etykiet. W pierwszej kontrolce wprowadzamy tekst wyświetlany w etykiecie na „Projekt kula”. W drugiej kontrolce wprowadzamy tekst wyświetlany „Podaj r”
3. Kontrolkę Text Field.
4. W trzeciej kontrolce etykiety usuwamy tekst wyświetlany w kontrolce.

Zmieniamy rozmiar czcionki we wszystkich kontrolkach na 24. Formularz powinien wyglądać jak na rysunku. Następnie oprogramujemy zdarzenie kliknięcia w przycisk.

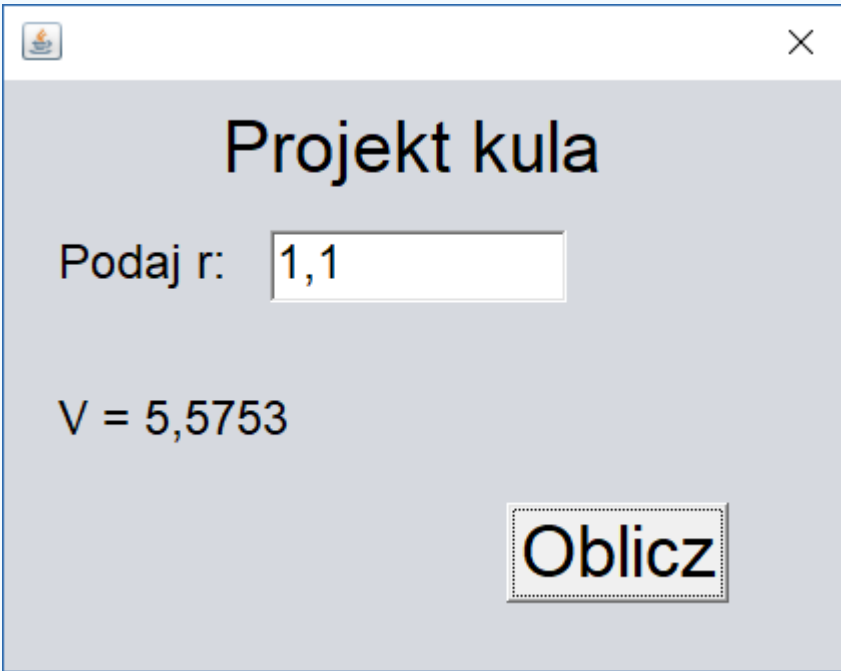
Projekt formularza

Projekt kula

Podaj r:

```
private void button1ActionPerformed
    (java.awt.event.ActionEvent evt) {
try{
    String s = textField1.getText();
    Scanner in = new Scanner(s);
    double r = in.nextDouble();
    if( r > 0.0){
        double v =4.0*Math.PI*r*r*r/3.0;
        NumberFormat f =
        NumberFormat.getNumberInstance();
        f.setMaximumFractionDigits(4);
        // f.setMinimumIntegerDigits(6);
        String s1 = f.format(v);
        label3.setText("V = " + s1);
    }
}
```

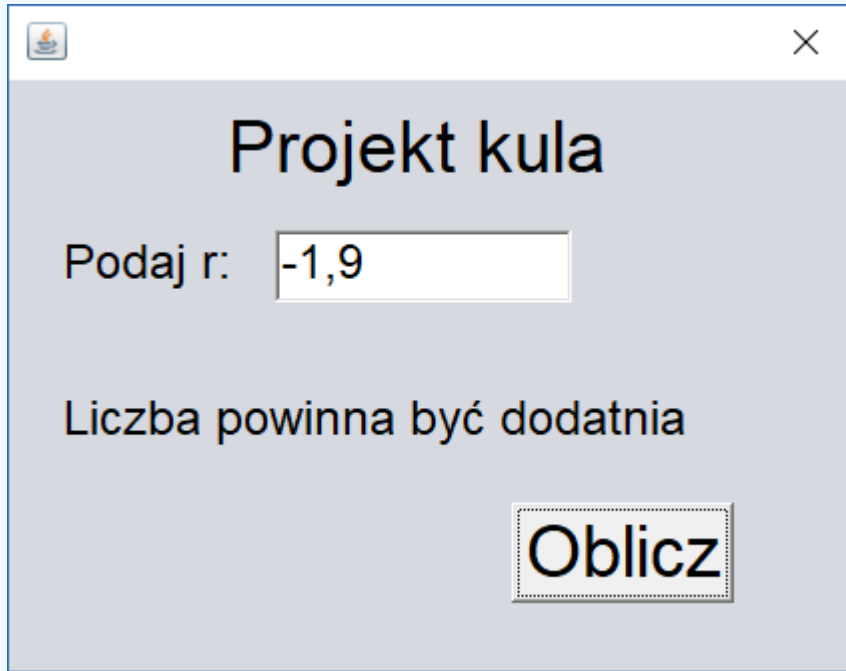
```
else label3.setText  
    ("Liczba powinna być dodatnia");  
}  
catch (Exception e) {  
    label3.setText ("Błąd konwersji");  
}  
}
```



Projekt kula

Podaj r:

$V = 5,5753$

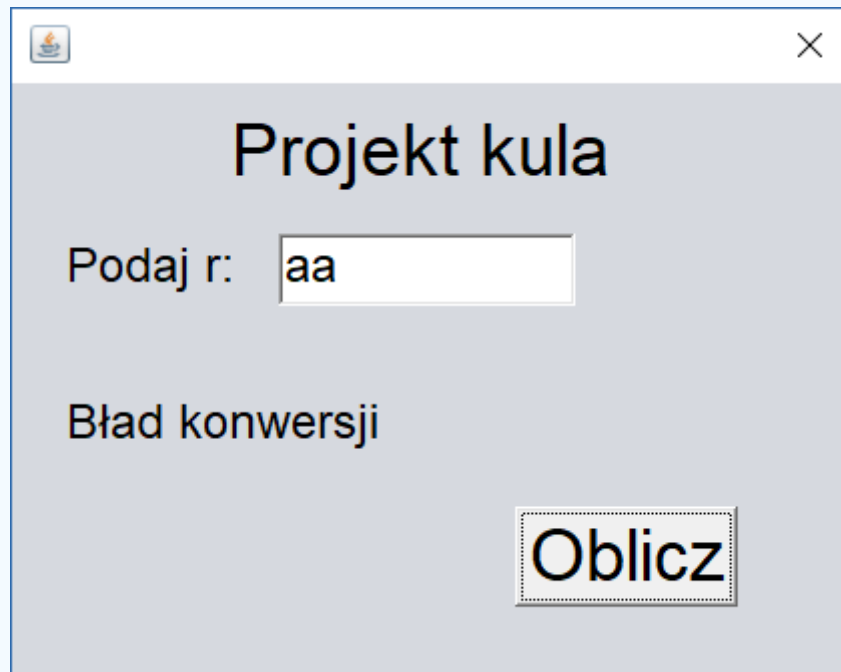


Projekt kula

Podaj r:

Liczba powinna być dodatnia

Sprawdzenie działania formularza



A screenshot of a Java Swing window titled "Projekt kula". The window has a standard title bar with a close button. The main content area is light gray. It contains the text "Projekt kula" in a large black font. Below it, the text "Podaj r:" is followed by a text input field containing the string "aa". Further down, the text "Bład konwersji" (Conversion error) is displayed. At the bottom right, there is a button labeled "Oblicz" (Calculate).

Projekt kula

Podaj r:

Bład konwersji

Oblicz

Projekt walec wykonany w JFrame Form

Zadanie. Należy zaprojektować formularz obliczający pole powierzchni całkowitej i objętość walca o zadanym promieniu podstawy i wysokości.

Rozwiązanie. Umieszczamy na powierzchni formularza pięć etykiet, dwie formatki TextField i przycisk. Do tekstu etykiety pierwsze wprowadzamy „Projekt walec”. Do tekstu etykiety drugiej wpisujemy „Podaj r:”. Do tekstu etykiety trzeciej wpisujemy „Podaj h:”. Z etykiety czwartej i piątej usuwamy tekst. Do tekstu wyświetlanego w przycisku wpisujemy „Oblicz”. Całość projektu powinna wyglądać tak jak na rysunku. Następnie oprogramujemy przycisk formularza.

Projekt formularza walec

Projekt walec

Podaj r:

Podaj h:

Oblicz

```
private void button1ActionPerformed
    (java.awt.event.ActionEvent evt) {
    label5.setText("");
    try{
        String s1 = textField1.getText();
        Scanner in1 = new Scanner(s1);
        double r = in1.nextDouble();
        if(r>0.0) {
            String s2 = textField2.getText();
            Scanner in2 = new Scanner(s2);
            double h = in2.nextDouble();
            if(h>0.0) {
                double P =
                    2.0 * Math.PI * r * ( r + h);
                double V = Math.PI * r * r * h;
```

```
        label4.setText( "P = " + P);  
        label5.setText("V = " + V);  
    }  
    else label4.setText  
    ("Wysokość powinna być dodatnia");  
}  
else label4.setText  
    ("Promień powinien być dodatni");  
}  
catch (Exception e) {  
    label4.setText("Błąd konwersji");  
}  
}
```

Sprawdzenie działania programu

Projekt walec

Podaj r:

Podaj h:

P = 15.205308443374602

V = 4.181459821928016

Projekt walec

Podaj r:

Podaj h:

Błąd konwersji

Projekt walec

Podaj r:

Podaj h:

Promień powinien być dodatni

Projekt walec

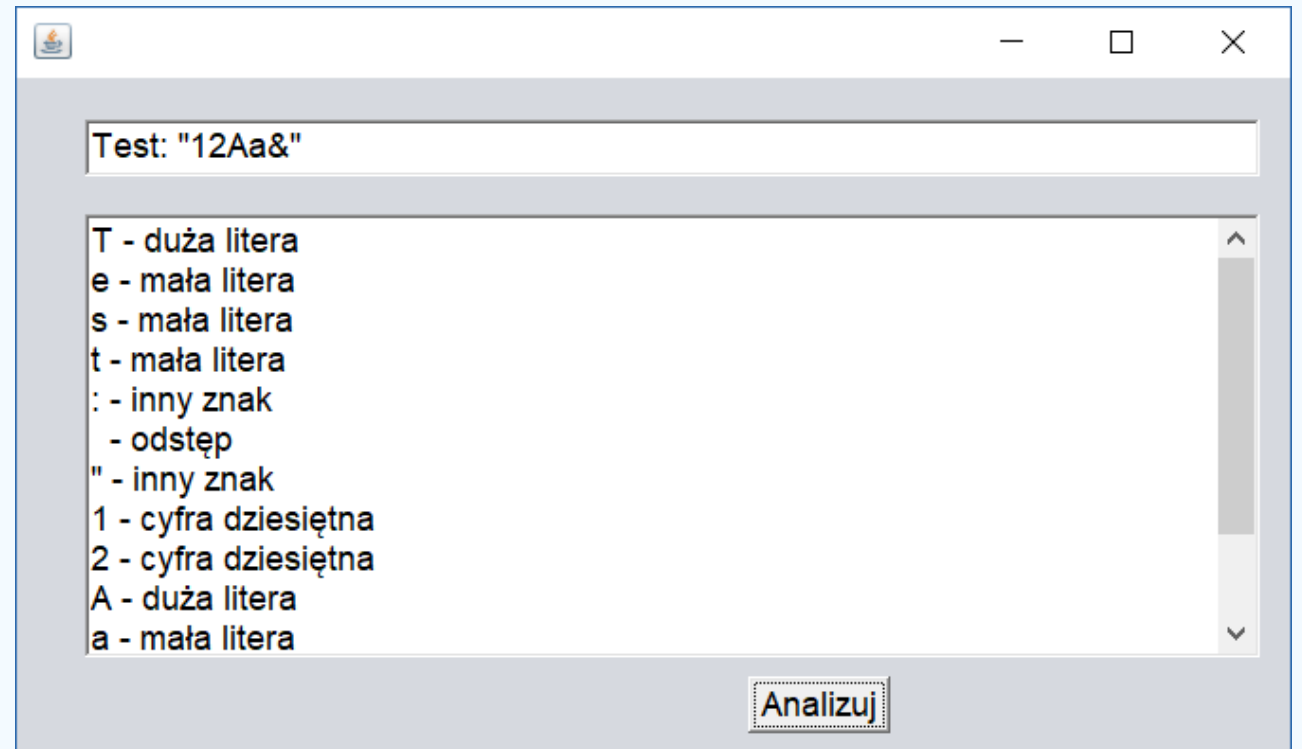
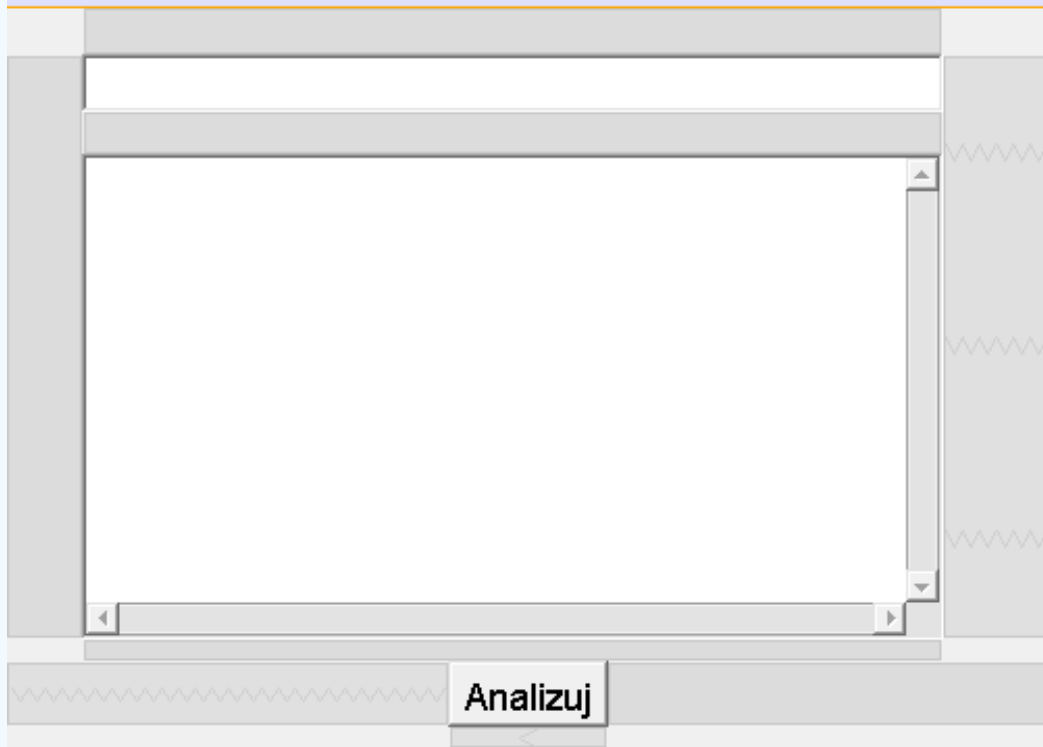
Podaj r:

Podaj h:

Wysokość powinna być dodatnia

Projekt analiza tekstu

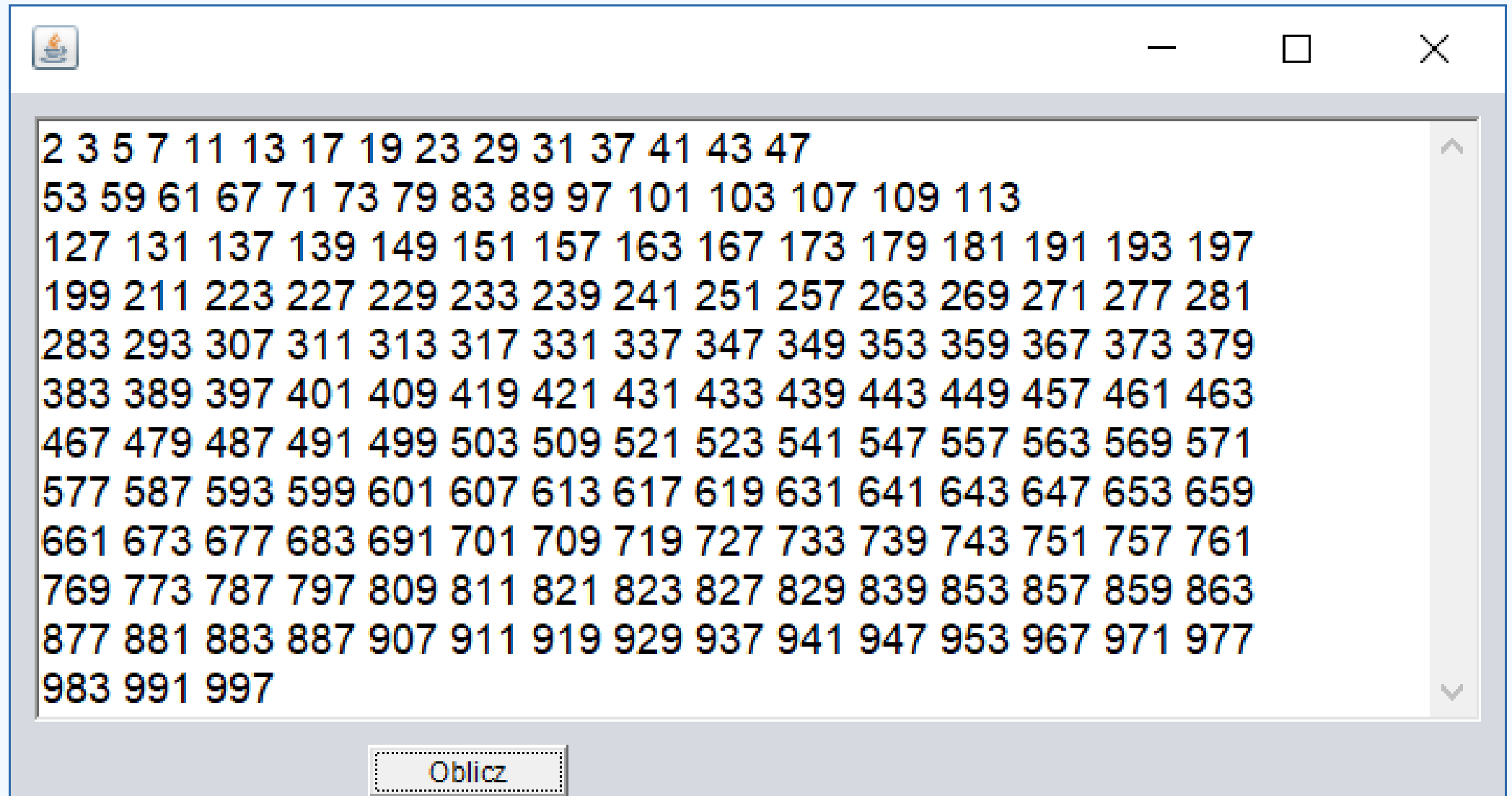
W projekcie zostały wykorzystane formatki: Text Field, Text Area i Button



```
private void button1ActionPerformed
    (java.awt.event.ActionEvent evt) {
    String tekst, linia = "";
    tekst = textField1.getText();
    char c;
    for(int i =0; i < tekst.length();i++) {
        c =tekst.charAt(i);
        linia += c + " - ";
        if (Character.isDigit(c)) linia +=
            "cyfra dziesiętna\r\n";
    }
}
```

```
else if (Character.isUpperCase(c))  
    linia += "duża litera\r\n";  
else if (Character.isLowerCase(c))  
    linia += "mała litera\r\n";  
else if (Character.isWhitespace(c))  
    linia += "odstęp\r\n";  
else linia += "inny znak\r\n";  
}  
textArea1.setText(linia);  
}
```

Projekt liczby pierwsze



```
private void button1ActionPerformed
    (java.awt.event.ActionEvent evt) {
    String s = ""; int max = 1000;
    int m = max + 1;
    Boolean[] tab = new Boolean[m];
    for (int i = 0; i < m; i++)
        tab[i] = true;
    for (int i = 2;
        i <= Math.sqrt((double)m); i++)
        if (tab[i])
            for (int j = 2 * i; j < m; j += i)
                tab[j] = false;
    int j = 0;
```

```
    for (int i = 2; i < m; i++) {  
        if (tab[i]) {  
            s += i + " ";  
            j++;  
            if (j%15==0) s+="\r\n";  
        }  
    }  
    textArea1.setText (s) ;  
}
```

JavaFX

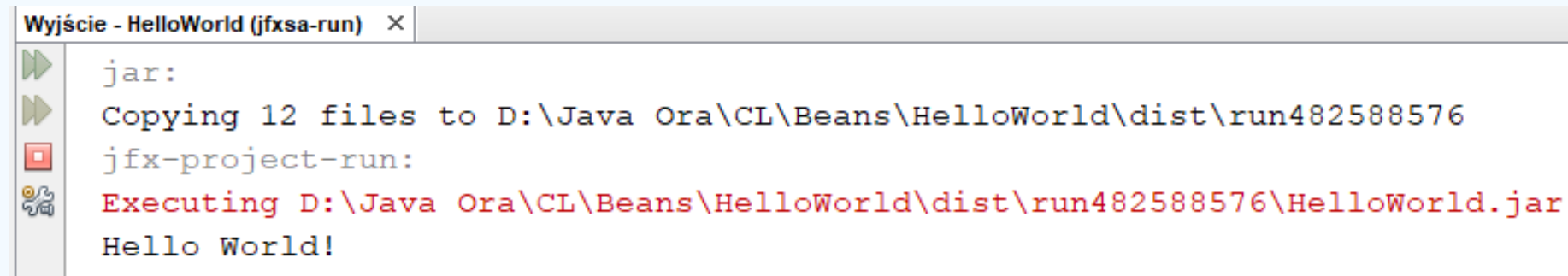
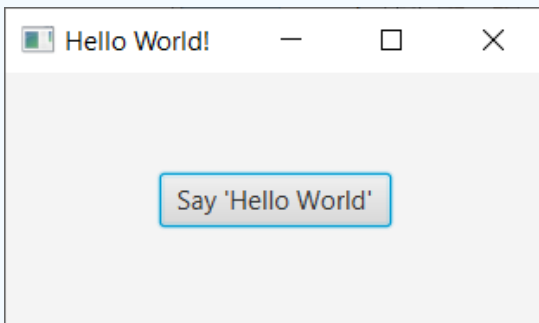
Aby utworzyć aplikację JavaFX w środowisku NetBeans wybieramy z menu **Plik - New Project**. W kategorii aplikacji **JavaFX** wybieramy **JavaFX Application** i klikamy w przycisk **Następny >**. Nazywamy projekt **HelloWorld** i umieszczamy projekt w żądanym katalogu. Następnie klikamy w zakładkę **Zakończ**.

```
package helloworld;  
  
import javafx.application.Application;  
import javafx.event.ActionEvent;  
import javafx.event.EventHandler;  
import javafx.scene.Scene;  
import javafx.scene.control.Button;  
import javafx.scene.layout.StackPane;  
import javafx.stage.Stage;
```

```
public class HelloWorld extends Application {  
    @Override  
    public void start(Stage primaryStage) {  
        Button btn = new Button();  
        btn.setText("Say 'Hello World'");  
        btn.setOnAction(new EventHandler<ActionEvent>() {  
            @Override  
            public void handle(ActionEvent event) {  
                System.out.println("Hello World!");  
            }  
        });  
        StackPane root = new StackPane();  
        root.getChildren().add(btn);  
        Scene scene = new Scene(root, 300, 250);  
        primaryStage.setTitle("Hello World!");  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

Aplikacja Witaj Świecie

Główna klasa aplikacji *JavaFX* jest rozszerzeniem klasy *javafx.application.Application*. Aplikacja *JavaFX* definiuje interfejs użytkownika za pomocą klasy ***Stage***, który jest argumentem metody *start()*. Klasa *Stage* jest kontenerem najwyższego poziomu, który zawiera kontrolkę przycisku z metodą obsługi zdarzenia. W *JavaFX* kontener reprezentuje hierarchiczny graf więzów. W przedstawionym przykładzie węzeł główny (ang. root node) jest obiektem klasy ***StackPane***. Obiekt klasy ***Scene*** jest wyświetlany w zadanym w pikselach rozmiarze. Oznacza to, że węzeł główny reprezentuje wyświetlany formularz, którego wymiary są możliwe do zmiany przez użytkownika.

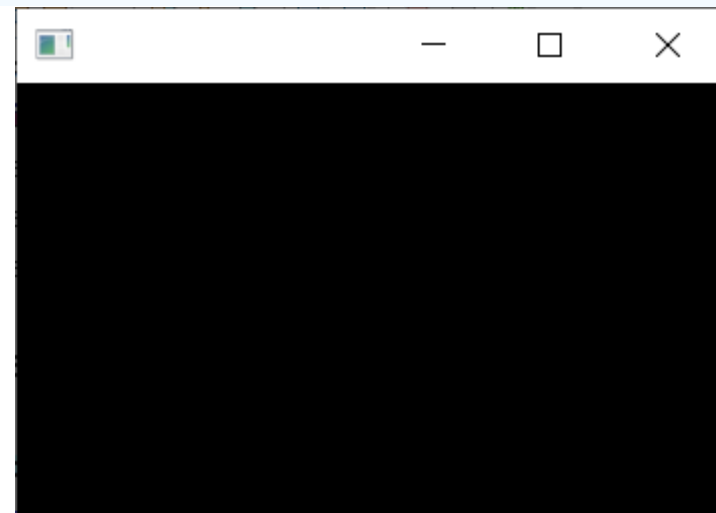


Aplikacja formularz w JavaFX

```
package colorblack;
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.paint.*;
import javafx.stage.*;

public class ColorBlack extends Application {
    @Override
    public void start(Stage primaryStage) {
        Group root = new Group();
        Scene scene = new Scene(root, 800, 600, Color.BLACK);
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```



Aplikacja koła

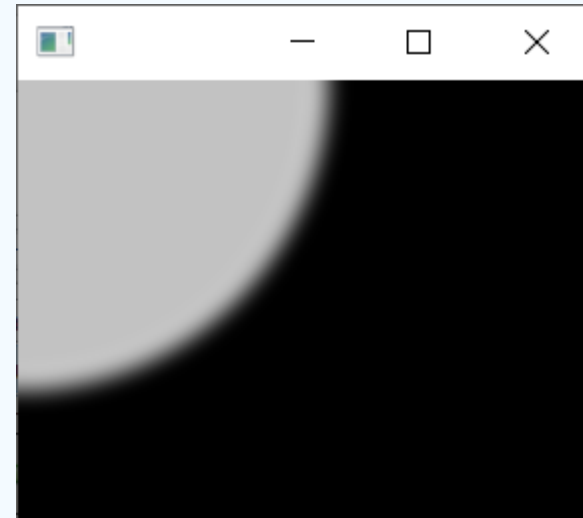
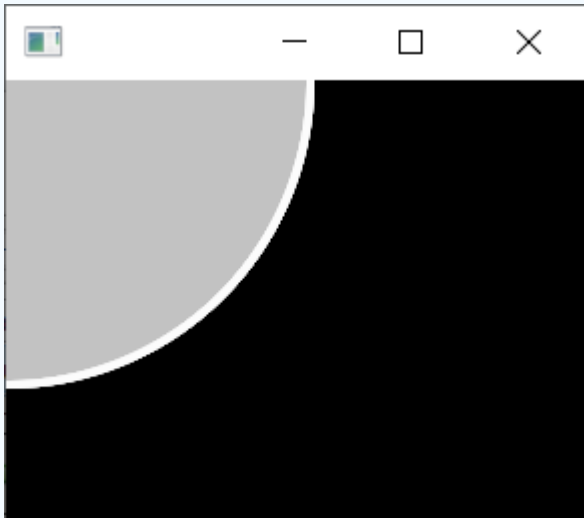
W przypadku przedstawionej aplikacji jako węzeł główny formularza został użyty węzeł grupy. Wielkość grupy jest podyktowana wielkością znajdujących się w niej węzłów. Jednak w przypadku większości aplikacji węzły powinny śledzić rozmiar formularza i zmieniać się po zmianie rozmiaru formularza. W takim przypadku jako główny używany jest węzeł układu o zmiennym rozmiarze.

```
package colorfulcircles;  
import javafx.application.Application;  
import javafx.scene.*;  
import javafx.scene.effect.*;  
import javafx.scene.paint.*;  
import javafx.scene.shape.*;  
import javafx.stage.*;
```

```
public class ColorfulCircles extends Application {  
    @Override  
    public void start(Stage primaryStage) {  
        Group root = new Group();  
        Scene scene = new Scene(root, 800, 600, Color.BLACK);  
        Group circles = new Group();  
        for (int i = 0; i < 30; i++) {  
            Circle circle = new Circle(150, Color.web("white", 0.05));  
            circle.setStrokeType(StrokeType.OUTSIDE);  
            circle.setStroke(Color.web("white", 0.16));  
            circle.setStrokeWidth(4);  
            circles.getChildren().add(circle);  
        }  
        root.getChildren().add(circles);  
        primaryStage.setScene(scene);  
        // circles.setEffect(new BoxBlur(10, 10, 3));  
        primaryStage.show();  
    }  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

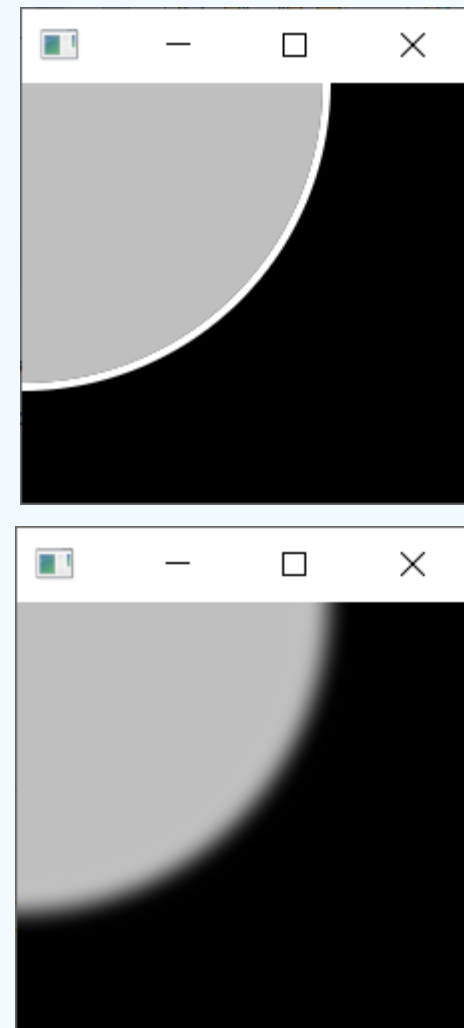
Aplikacja koła

- Przedstawiony kod programu tworzy grupę o nazwie *circles*, a następnie wykorzystuje pętlę, aby dodać do grupy 30 kręgów. Każde koło ma promień 150, kolor wypełnienia biały z poziom krycia w 5 procentach, co oznacza, że są w większości przezroczyste.
- Aby utworzyć ramkę wokół kręgów, kod zawiera klasę *StrokeType*. Typ obrysu *OUTSIDE* oznacza, że granica okręgu jest przedłużona poza jego wnętrze o wartość *StrokeWidth* którą jest 4. Kolor obrysu jest biały, a poziom krycia to 16 procent, dzięki czemu jest jaśniejszy niż kolor kół.
- Ostatni wiersz dodaje grupę okręgów do węzła głównego.
- Efekt rozmycia ramki do okręgów jest nieostry. Można go wyostrzyć zdejmując komentarz w programie w instrukcji przed wierszem *primaryStage.show()*. Ta instrukcja ustawia promień rozmycia na 10 pikseli szerokości i wysokości, a iterację rozmycia na 3, tak aby była zbliżona do rozmycia Gaussa. Ta technika rozmycia daje efekt wygładzenia krawędzi okręgów, jak pokazano na rysunku .



Uproszczona aplikacja koła

```
@Override
public void start(Stage primaryStage) {
    Group root = new Group();
    Scene scene = new Scene(root, 800, 600, Color.BLACK);
    Group circles = new Group();
    Circle circle = new Circle(150, Color.web("white", 0.75));
    circle.setStrokeType(StrokeType.OUTSIDE);
    circle.setStroke(Color.web("white"));
    circle.setStrokeWidth(4);
    circles.getChildren().add(circle);
    root.getChildren().add(circles);
    primaryStage.setScene(scene);
    // circles.setEffect(new BoxBlur(10, 10, 3));
    primaryStage.show();
}
```



Kolorowy formularz

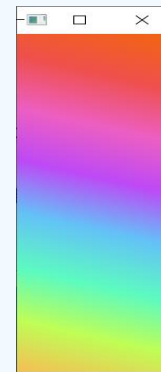
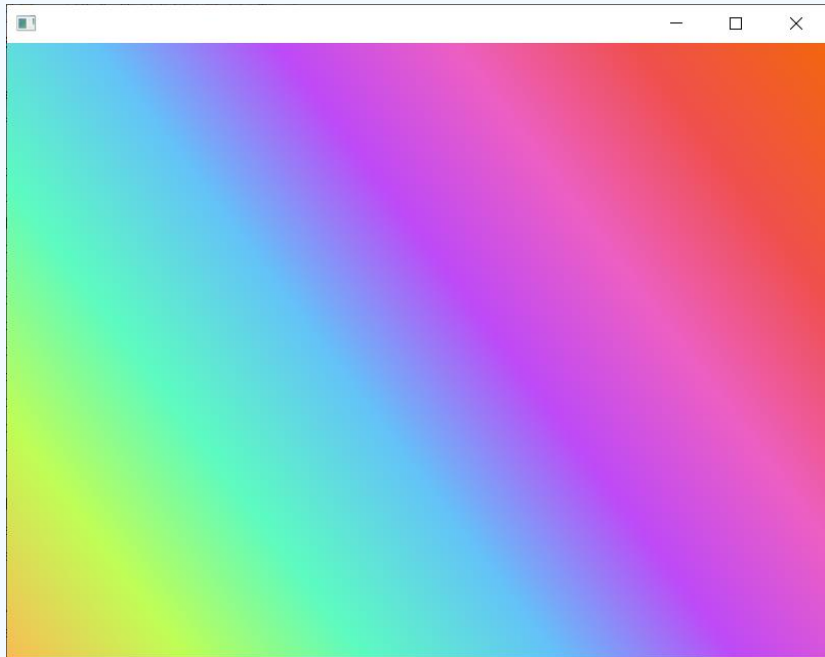
Przedstawiony poniżej kod tworzy prostokąt o nazwie **colors**. Prostokąt ma taką samą szerokość i wysokość jak formularz i jest wypełniony gradientem liniowym, który zaczyna się w lewym dolnym rogu (0, 1) i kończy w prawym górnym rogu (1, 0). Wartość **true** oznacza, że gradient jest proporcjonalny do prostokąta i **NO_CYCLE** wskazuje, że cykl kolorów się nie powtórzy. Tablica **Stop[]** wskazuje, jaki kolor gradientu powinien być w danym miejscu. Następne dwa wiersze kodu dostosowują gradient liniowy w miarę zmiany rozmiaru formularza, wiążąc szerokość i wysokość prostokąta z szerokością i wysokością sceny.

```
package colorfull;
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.paint.*;
import javafx.scene.shape.*;
import javafx.stage.*;
```

```
public class ColorFull extends Application {  
    @Override  
    public void start(Stage primaryStage) {  
        Group root = new Group();  
        Scene scene = new Scene(root, 800, 600);  
        Rectangle colors = new Rectangle(scene.getWidth(), scene.getHeight(),  
            new LinearGradient(0f, 1f, 1f, 0f, true, CycleMethod.NO_CYCLE,  
                new Stop[]{  
                    new Stop(0, Color.web("#f8bd55")),  
                    new Stop(0.14, Color.web("#c0fe56")),  
                    new Stop(0.28, Color.web("#5dfbc1")),  
                    new Stop(0.43, Color.web("#64c2f8")),  
                    new Stop(0.57, Color.web("#be4af7")),  
                    new Stop(0.71, Color.web("#ed5fc2")),  
                    new Stop(0.85, Color.web("#ef504c")),  
                    new Stop(1, Color.web("#f2660f")), }));  
        colors.widthProperty().bind(scene.widthProperty());  
        colors.heightProperty().bind(scene.heightProperty());  
        root.getChildren().add(colors);  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
}
```

Kolorowy formularz

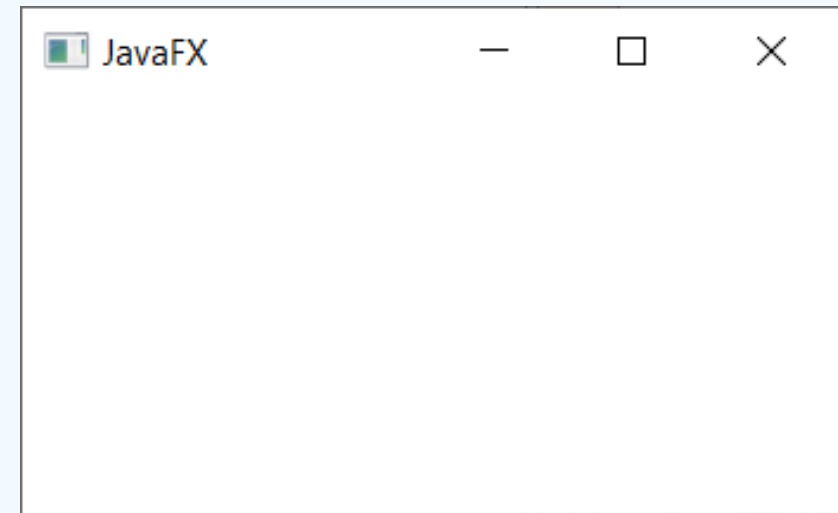
```
public static void main(String[] args) {  
    launch(args);  
}  
}
```



Tworzenie formularza w JavaFX

```
package logina;
import javafx.application.Application;
import javafx.event.*;
import javafx.geometry.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.paint.*;
import javafx.scene.text.*;
import javafx.stage.*;
public class LoginA extends Application {
    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("JavaFX");

        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```



Siatka formularza GridPane

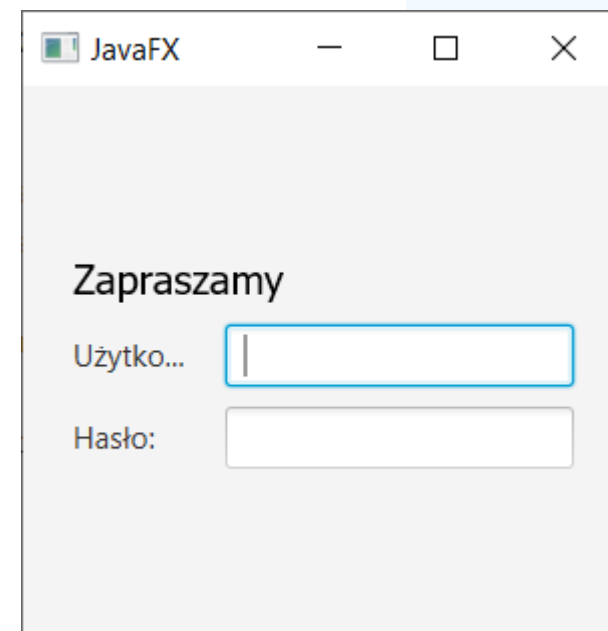
Siatka **GridPane** umożliwia utworzenie siatki wierszy i kolumn, w której można розміścić kontrolki. Tworzony obiekt klasy **GridPane** został przypisany do zmiennej **grid**. Właściwość wyrównania zmienia domyślne położenie siatki z lewego górnego rogu sceny do środka. Właściwości przerwy zarządzają odstępami między wierszami i kolumnami, podczas gdy właściwość **padding** zarządza przestrzenią wokół krawędzi panelu siatki.

```
public void start(Stage primaryStage) {  
    primaryStage.setTitle("JavaFX");  
  
    GridPane grid = new GridPane();  
    grid.setAlignment(Pos.CENTER);  
    grid.setHgap(10);  
    grid.setVgap(10);  
    grid.setPadding(new Insets(25, 25, 25, 25));  
    Scene scene = new Scene(grid, 300, 275);  
    primaryStage.setScene(scene);  
  
    primaryStage.show();  
}
```

Tekst, etykieta i pole tekstowe

Pierwsze linie kodu tworzą obiekt ***Tekst***, który jest wstawiany do siatki pod indeks (0, 0). Kolejne wiersze tworzą obiekt etykiety z tekstem oraz obiekt pola tekstowego, który można edytować. Pole hasła i etykieta są tworzone i dodawane do panelu siatki w podobny sposób. Numeracja kolumn i wierszy rozpoczyna się od indeksu (0, 0) i jest odwrotny niż indeksy tablicy dwuwymiarowej – indeks pierwszy kolumna. drugi wiersz .

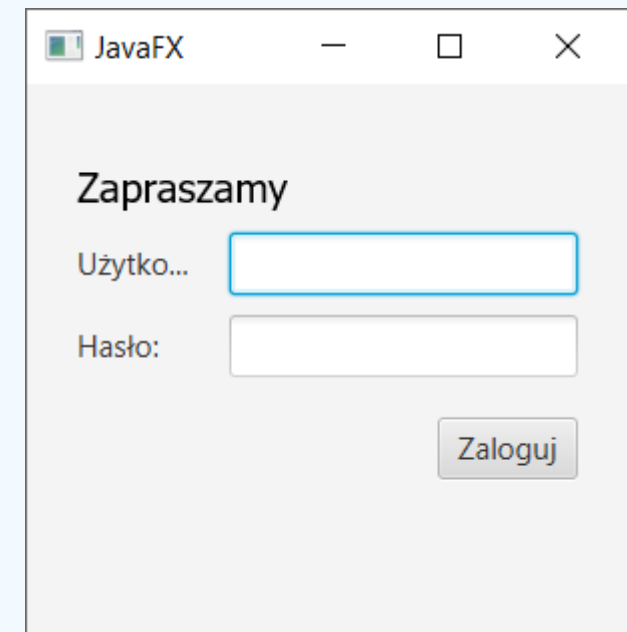
```
Text scenetitle = new Text("Zapraszamy");
scenetitle.setFont(Font.font("Tahoma", FontWeight.NORMAL, 20));
grid.add(scenetitle, 0, 0, 2, 1);
Label userName = new Label("Użytkownik:");
grid.add(userName, 0, 1);
TextField userTextField = new TextField();
grid.add(userTextField, 1, 1);
Label pw = new Label("Hasło:");
grid.add(pw, 0, 2);
PasswordField pwBox = new PasswordField();
grid.add(pwBox, 1, 2);
```



Przycisk i etykieta na komunikaty

Najpierw tworzony jest przycisk z etykietą **Zaloguj**. Kolejne linie tworzą panel układu o nazwie **hbBtn** z odstępami 10 pikseli. Okienko ustawia wyrównanie przycisku, które różni się **HBoxod** wyrównaniem zastosowanego do innych kontrolek w okienku siatki. Właściwość **alignment** ma wartość **Pos.BOTTOM_RIGHT**, co oznacza położenie węzła na dole dosuniętego do prawej krawędzi. Przycisk zostanie dodany jako element podrzędny **Hboxpanelu** i **HBoxpanel** zostaje dodany do siatki w kolumnie 1, wiersz 4.

```
Button btn = new Button("Zaloguj");
HBox hbBtn = new HBox(10);
hbBtn.setAlignment(Pos.BOTTOM_RIGHT);
hbBtn.getChildren().add(btn);
grid.add(hbBtn, 1, 4);
final Text actiontarget = new Text();
grid.add(actiontarget, 1, 6);
```

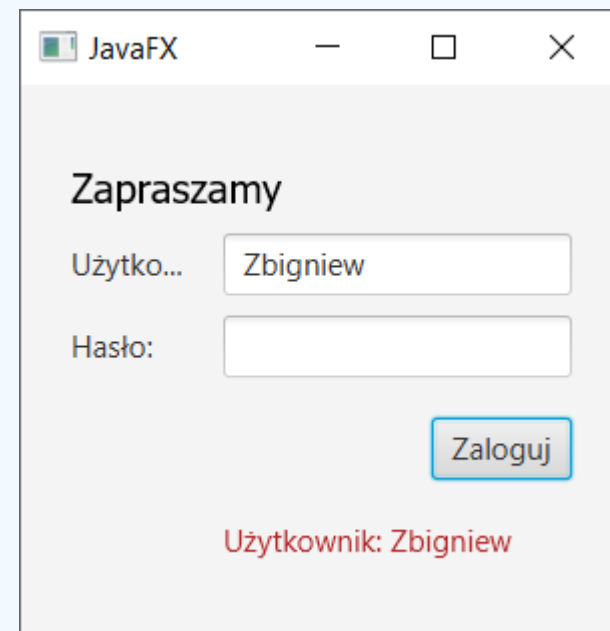


Obsługa zdarzenia przycisku

```
btn.setOnAction(new EventHandler<ActionEvent>() {  
    @Override  
    public void handle(ActionEvent e) {  
        actiontarget.setFill(Color.FIREBRICK);  
        actiontarget.setText("Użytkownik: "+userTextField.getText());  
    }  
});
```

Albo alternatywny kod za pomocą operatora lambda.

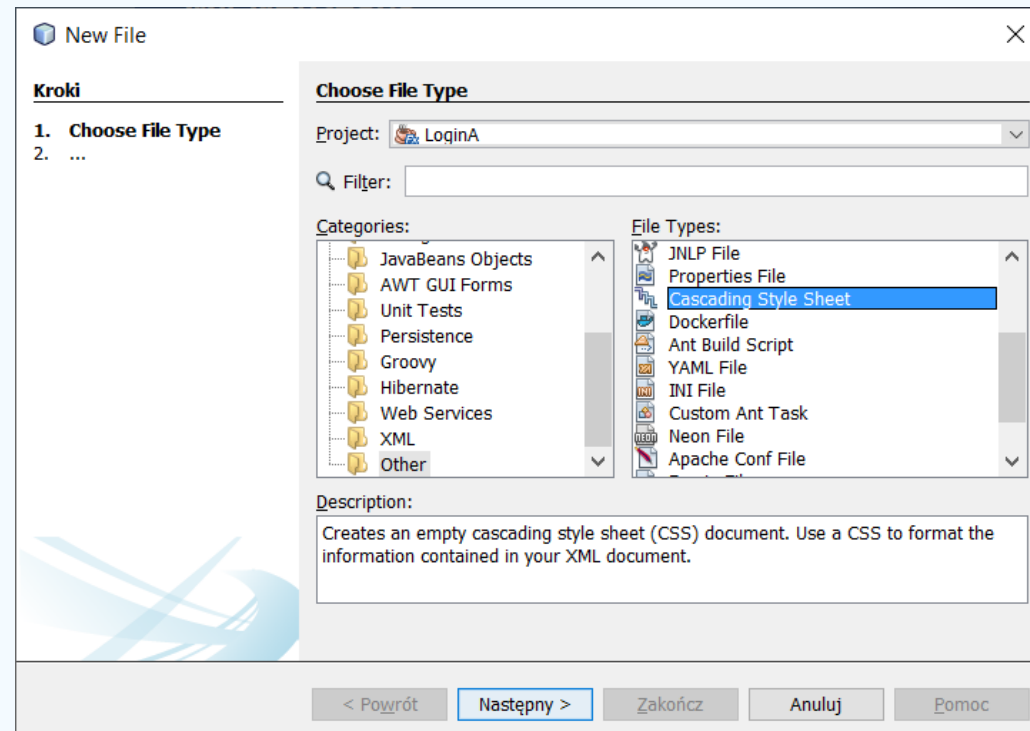
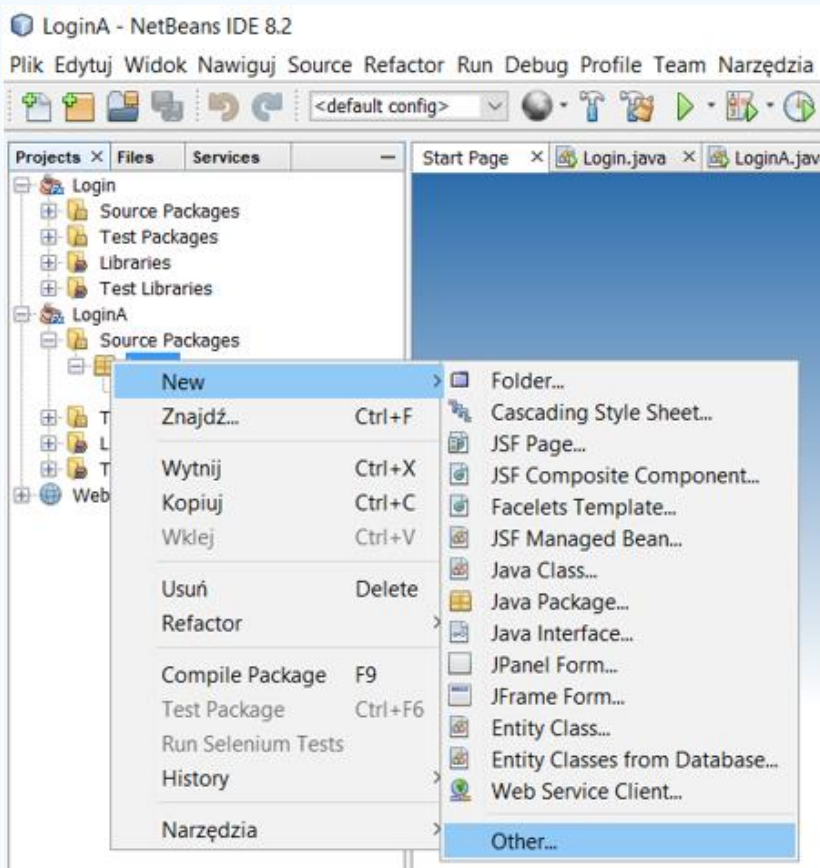
```
btn.setOnAction((ActionEvent e) -> {  
    actiontarget.setFill(Color.FIREBRICK);  
    actiontarget.setText("Użytkownik: "+userTextField.getText());  
});
```



Obraz jako tło formularza

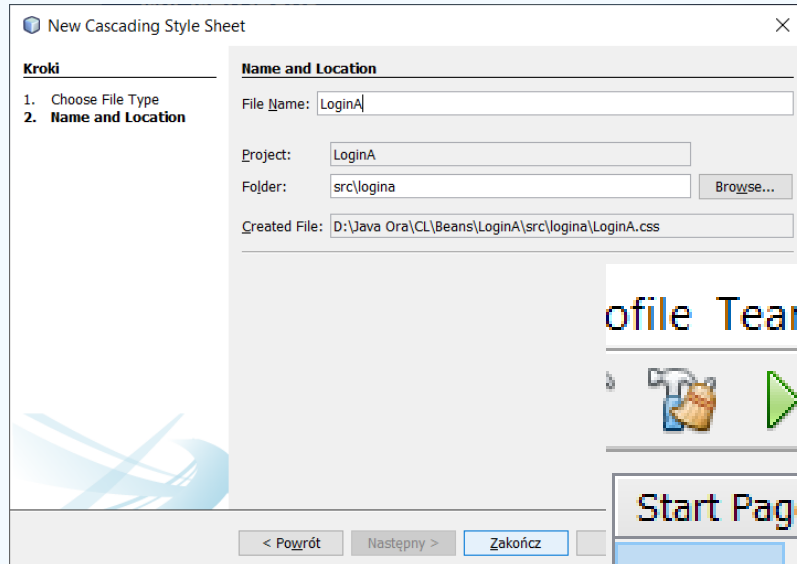
Tworzymy plik CSS i zapisujemy go w katalogu aplikacji. Następnie dodajemy kod w aplikacji o dowiązaniu kaskadowego arkusza stylów. W tym celu wykonujemy polecenia:

1. W oknie Projektu NetBeans IDE rozwijamy węzeł projektu, a następnie węzeł katalogu.
2. Kliknij prawym przyciskiem myszy folder o nazwie projektu w katalogu **Source Packages** i wybierz **New** , a następnie **Other...** .
3. W oknie dialogowym wybieramy opcję **Other** , a następnie **Cascading Style Sheet** i kliknij **Następny>** .

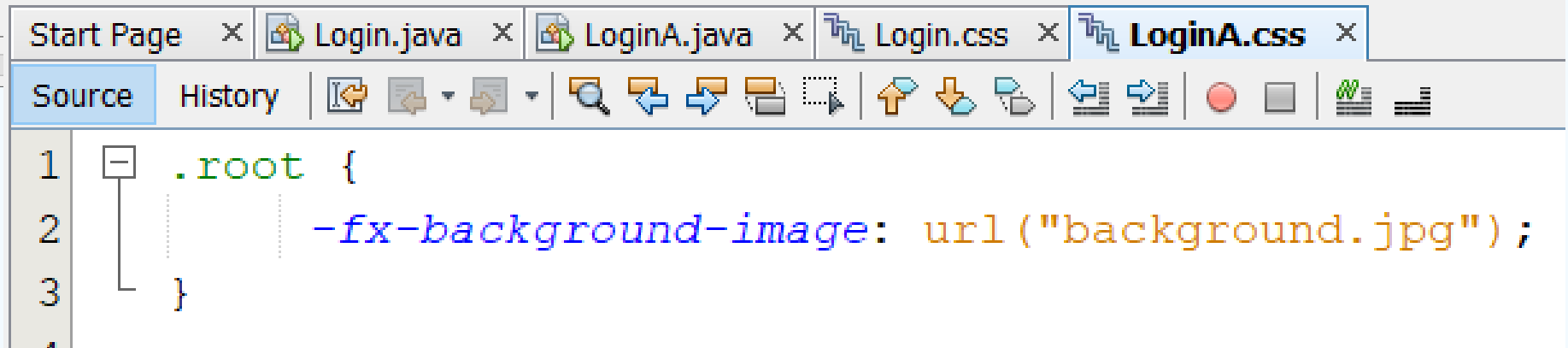
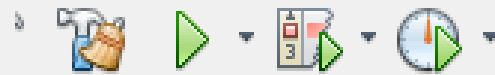


4. W kreatorze kaskadowego arkusza stylów wpisujemy nazwę pliku taką samą jak nazwa projektu i klikamy w przycisk **Zakończ** >.

5. W edytorze kaskadowego arkusza stylów wpisujemy kod jak na rysunku.



ofile Team Narzędzia Okno Pomoc

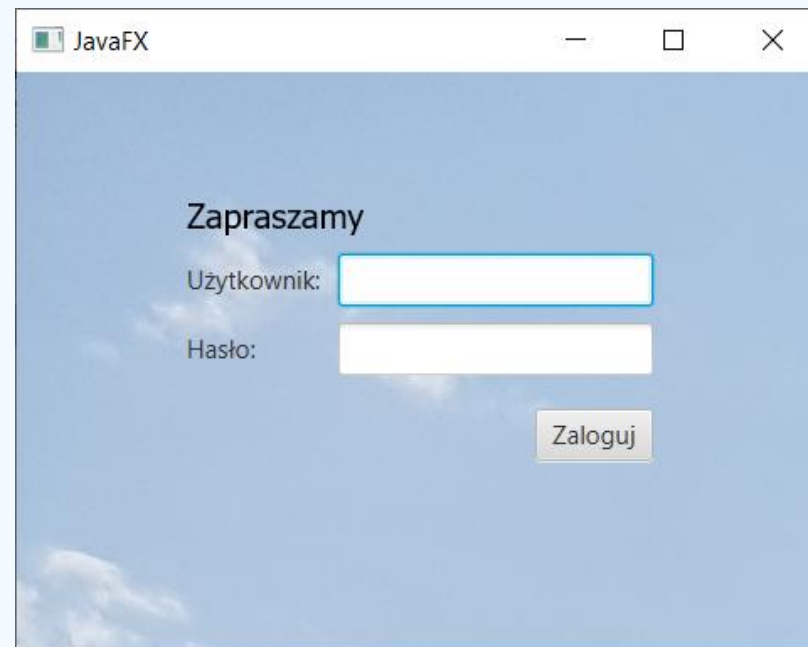


6. W pliku z kodem projektu po instrukcji `primaryStage.setScene(scene);` i przed kolejną instrukcją wpisujemy instrukcję połączenia formularza z kaskadowym arkuszem stylów tak jak na rysunku.

```
primaryStage.setScene(scene);  
  
scene.getStylesheets().add  
    (LoginA.class.getResource("LoginA.css").toExternalForm());
```

7. W katalogu projektu i pod folderze `src\nazwa_projektu` wklejamy obraz o nazwie `background.jpg` i uruchamiamy projekt.

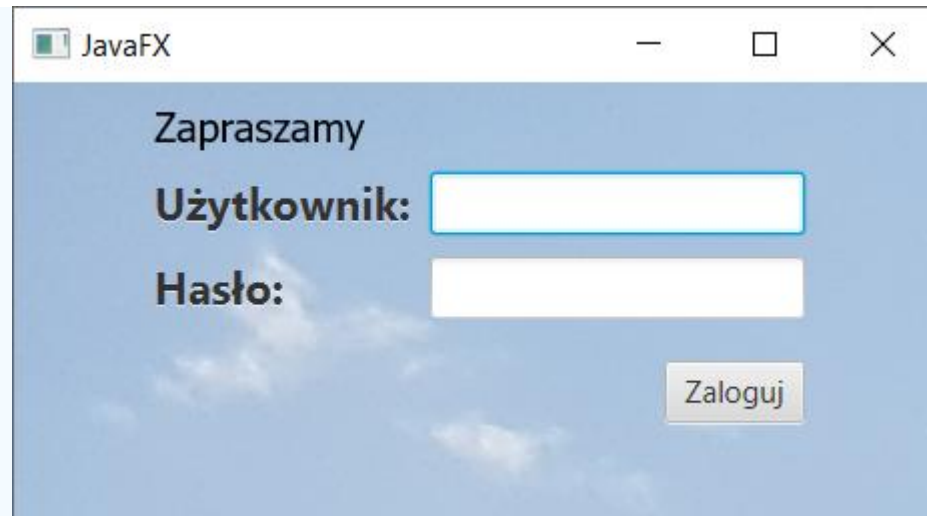
owy (D:) > Java Ora > CL > Beans > LoginA > src > logina		
Nazwa	Data modyfikacji	
 background.jpg	14.05.2018 10:05	
 LoginA.css	18.09.2022 12:52	
 LoginA.java	17.09.2022 17:22	



Styl etykiet

Ten przykład zwiększa rozmiar i grubość czcionki w etykietach oraz stosuje cień w kolorze szarym (#333333). Zadaniem cienia jest dodanie kontrastu między ciemnoszarym tekstem a jasnoszarym tłem. Do pliku CSS należy dodać kod:

```
.label {  
    -fx-font-size: 22px;  
    -fx-font-weight: bold;  
    -fx-text-fill: #333333;  
    -fx-effect: dropshadow( gaussian , rgba(255,255,255,0.5) , 0,0,0,1 );  
}
```



Styl tekstu

W celu sformatowania tekstu dwóch obiektów formularz scenetitle i actiontarget usuwamy z kodu programu dwie instrukcje:

```
scenetitle.setFont(Font.font("Tahoma", FontWeight.NORMAL, 20));
```

```
actiontarget.setFill(Color.FIREBRICK);
```

Następnie wstawiamy identyfikatory każdego węzła korzystając z metody ***setId()*** klasy ***Node***.

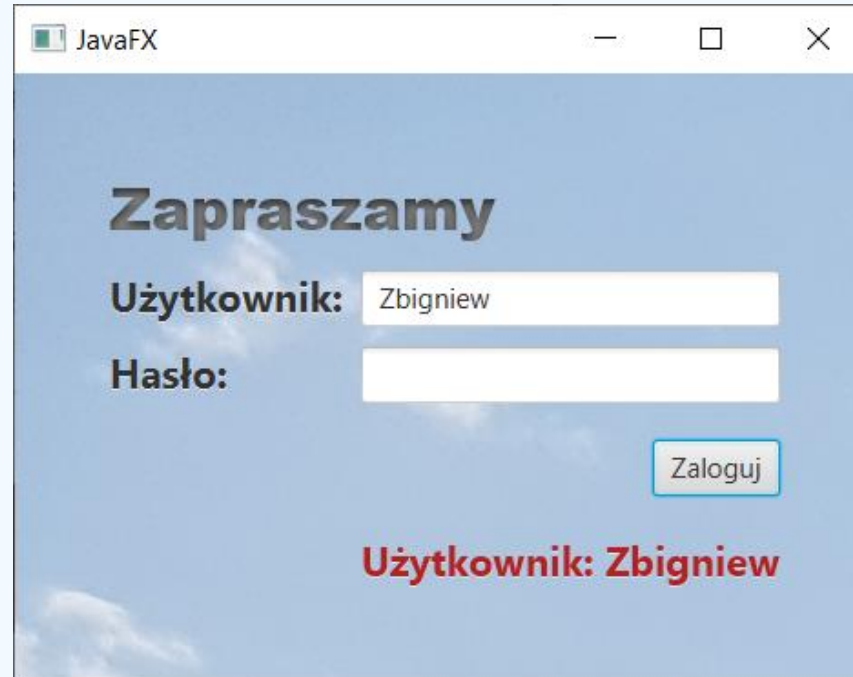
```
scenetitle.setId("welcome-text");
```

```
actiontarget.setId("actiontarget");
```

W pliku CSS definiujemy właściwości stylu dla identyfikatorów ***welcome-text*** i ***actiontarget*** jak pokazano na rysunku.

```
#welcome-text {  
    -fx-font-size: 32px;  
    -fx-font-family: "Arial Black";  
    -fx-fill: #818181;  
    -fx-effect: innershadow( three-pass-box , rgba(0,0,0,0.7) , 6, 0.0 , 0 , 2 );  
}  
  
#actiontarget {  
    -fx-font-size: 22px;  
    -fx-fill: FIREBRICK;  
    -fx-font-weight: bold;  
    -fx-effect: dropshadow( gaussian , rgba(255,255,255,0.5) , 0,0,0,1 );  
}
```

Stylizacja przycisku



Następnym krokiem jest nadanie stylu przycisku, dzięki czemu zmieni on styl, gdy użytkownik najedzie na niego myszą. Ta zmiana da użytkownikom wskazówkę, że przycisk jest interaktywny, co jest standardową praktyką projektową. W tym celu uzupełniamy kod pliku CSS.

Formularz logowania

```
.przycisk {  
    -fx-text-fill: biały;  
    -fx-font-family: "Arial Narrow";  
    -fx-font-weight: pogrubienie;  
    -fx-background-color: liniowy-gradient (#61a2b1, #2A5058);  
    -fx-effect: dropshadow( trzyprzebiegowe pola ,  
                           rgba(0,0,0,0.6) , 5, 0.0 , 0 , 1 );  
}  
  
.przycisk:najedź {  
    -fx-background-color: liniowy-gradient (#2A5058, #61a2b1);  
}
```

